DEEP LEARNING

GITAM

DEEMED TO BE UNIVERSITY

Course Outcome 1

Understand the role of neural networks and its various applications.

Contents

- Artificial Neuron
- Feed Forward Networks
- Gradient descent
- Back propagation
- Regularization techniques
- Norm penalties as constrained optimization

Reference

- Chapter 1, 2

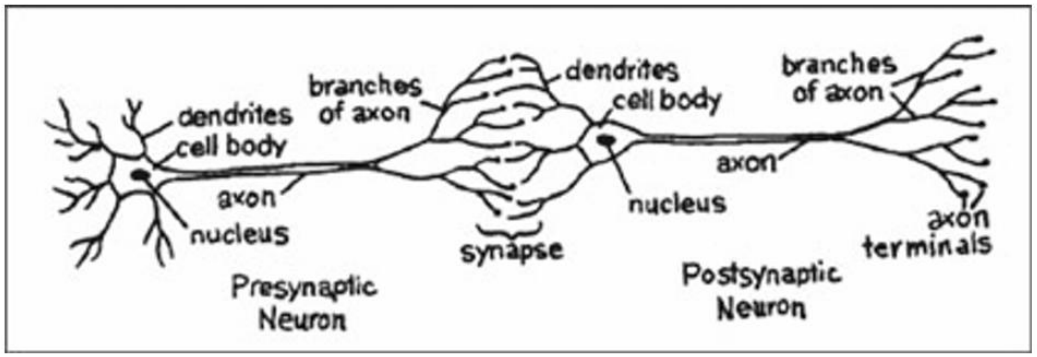
Charu C. Aggarwal, “Neural Networks and Deep Learning”, Springer International Publishing AG

- Deep Learning- Charu Aggarwal

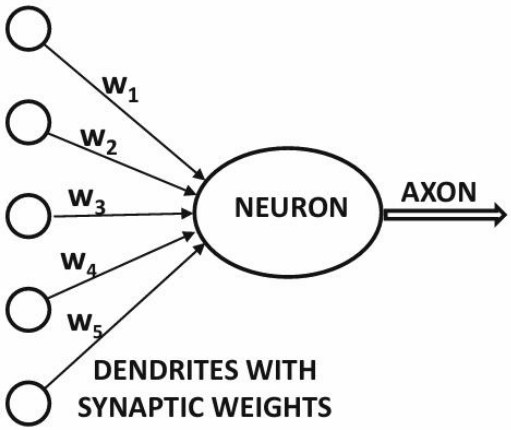
https://www.youtube.com/playlist?list=PLLo1RD8Vbbb_6gCyqxG_qzCLOj9EKubw7

- Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, MIT Press, 2016

Neuron

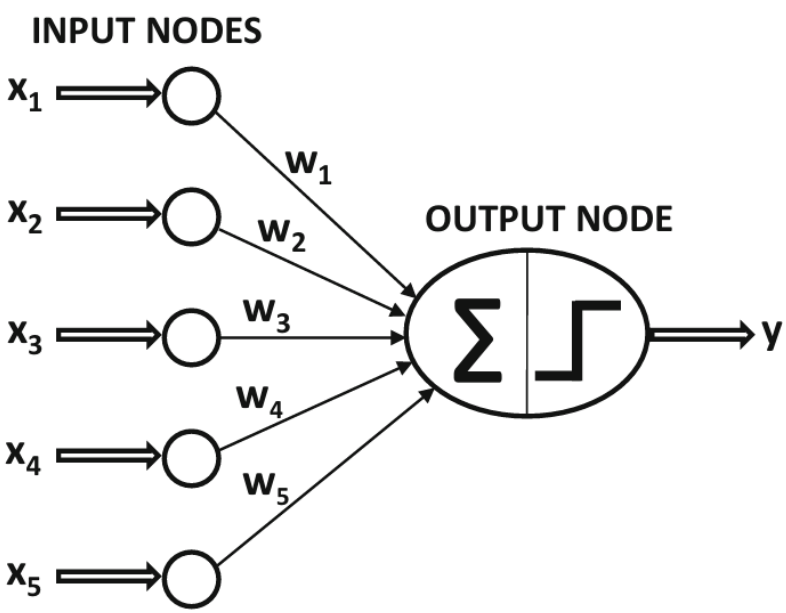


(a) Biological neural network

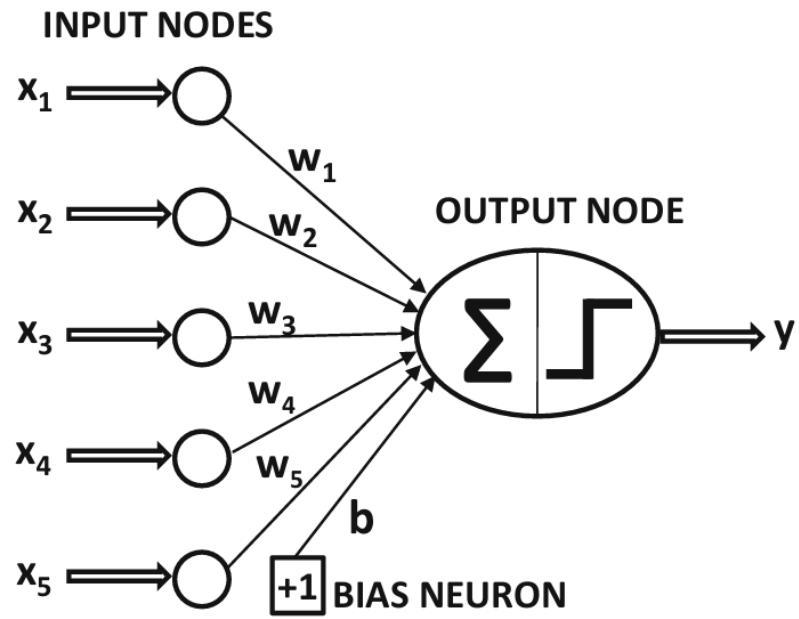


(b) Artificial neural network

Perceptron



(a) Perceptron without bias

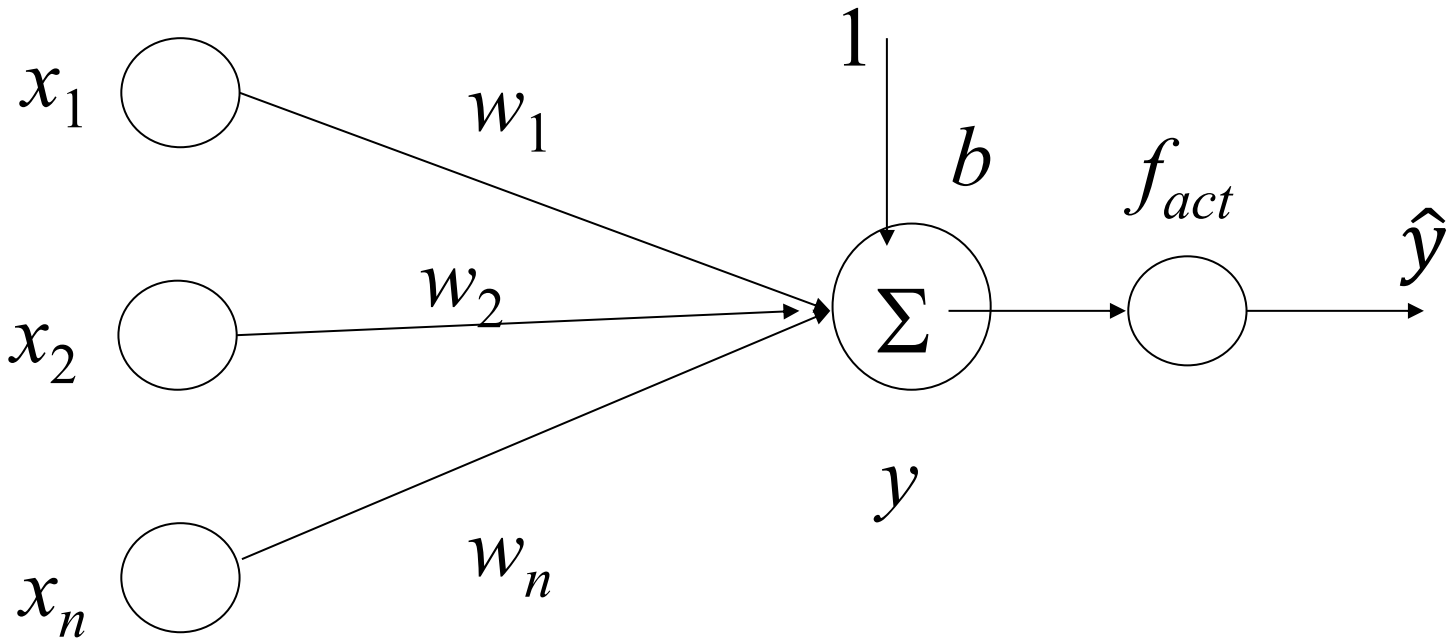


(b) Perceptron with bias

The simplest neural network is referred to as the perceptron. This neural network contains a single input layer and an output node.

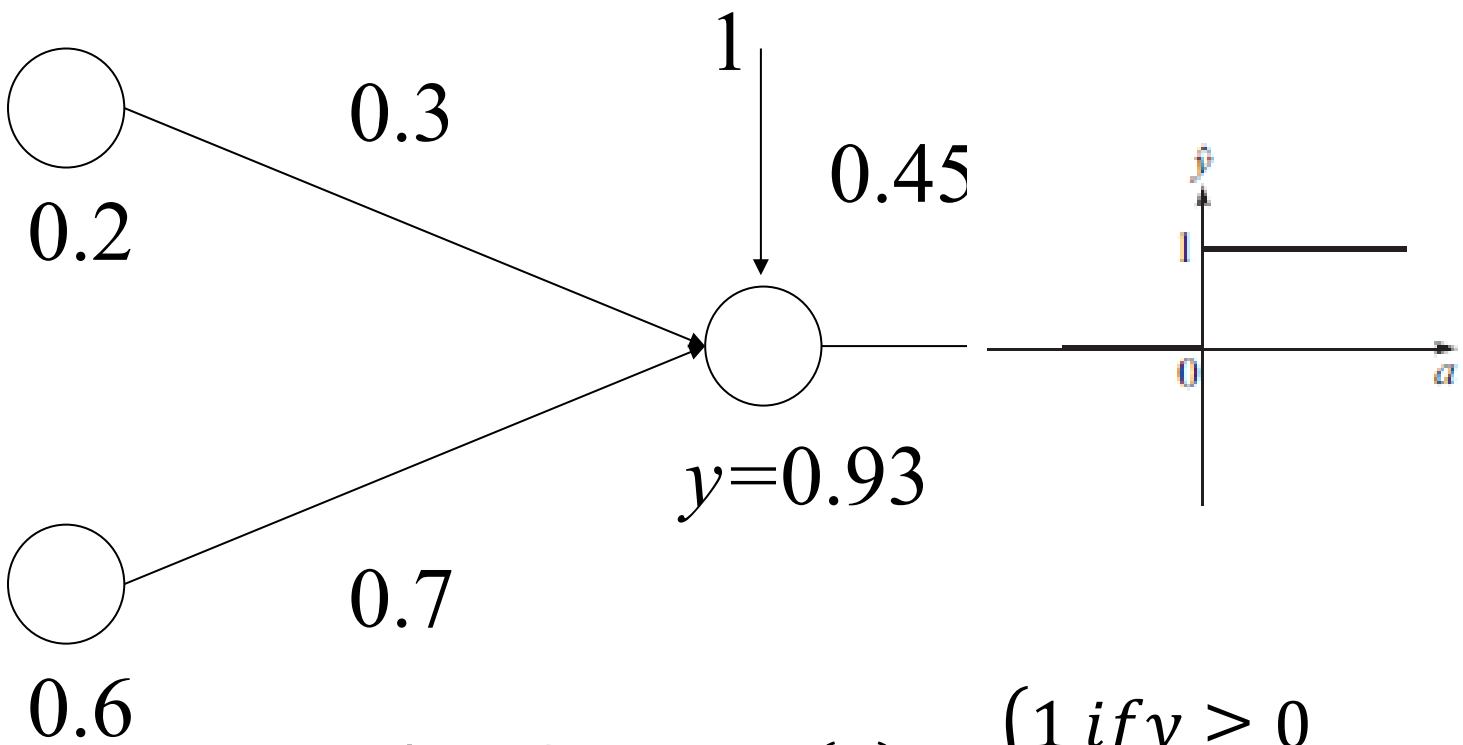
Artificial Neuron

$$y = w_1\, x_1 + w_2\, x_2 \ldots + w_n\, x_n + b$$



Numerical Example 1

Calculate the output assuming Binary step activation function



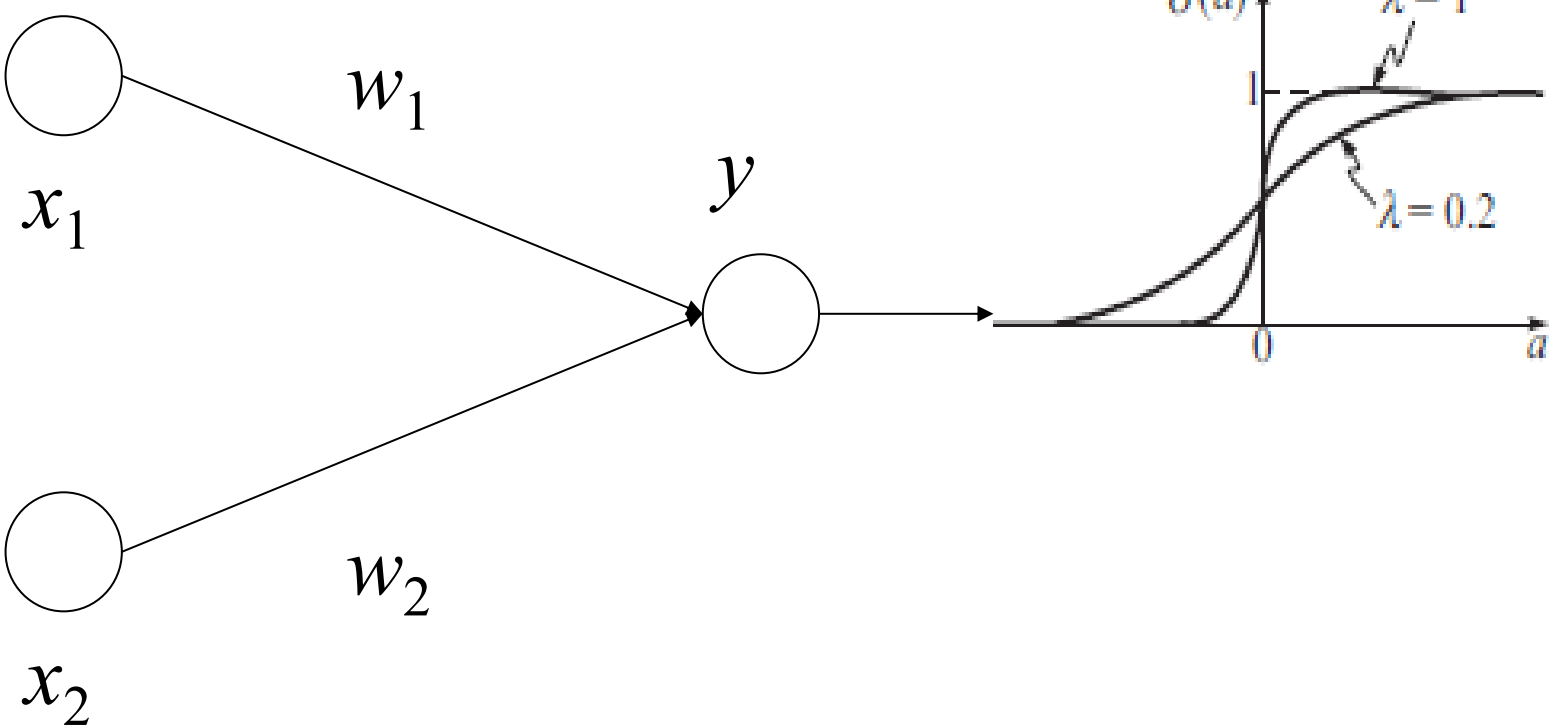
$$\hat{y} = f_{activation}(y) = \begin{cases} 1 \; if \; y \geq 0 \\ 0 \; if \; y < 0 \end{cases}$$

Artificial Neuron model

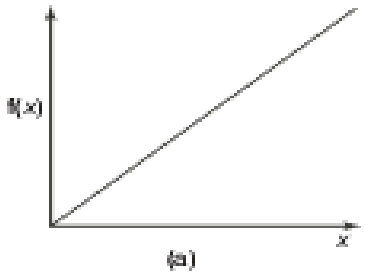
Sigmoidal function

$$\hat{y} = f_{activation}(y) = \frac{1}{1 + e^{-\lambda y}}$$

λ=1 is Binary sigmoidal function

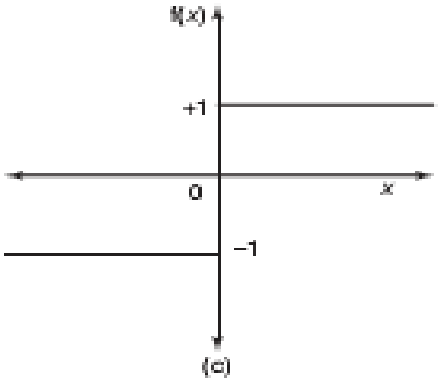


ACTIVATION FUNCTIONS



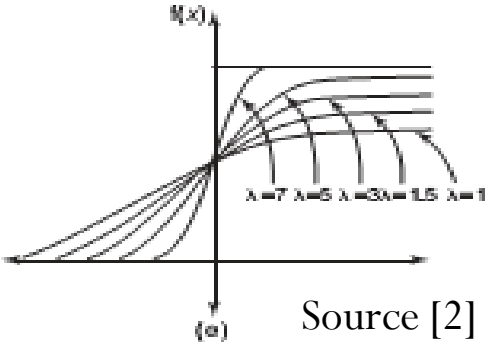
(A)Identity/linear

$$f_{act}(x) = \begin{cases} x \; if x \geq 0 \\ 0 \; if x < 0 \end{cases}$$



(B) Bipolar Binary step

$$f_{act}(x) = \begin{cases} 1 \; if x \geq 0 \\ -1 \; if x < 0 \end{cases}$$



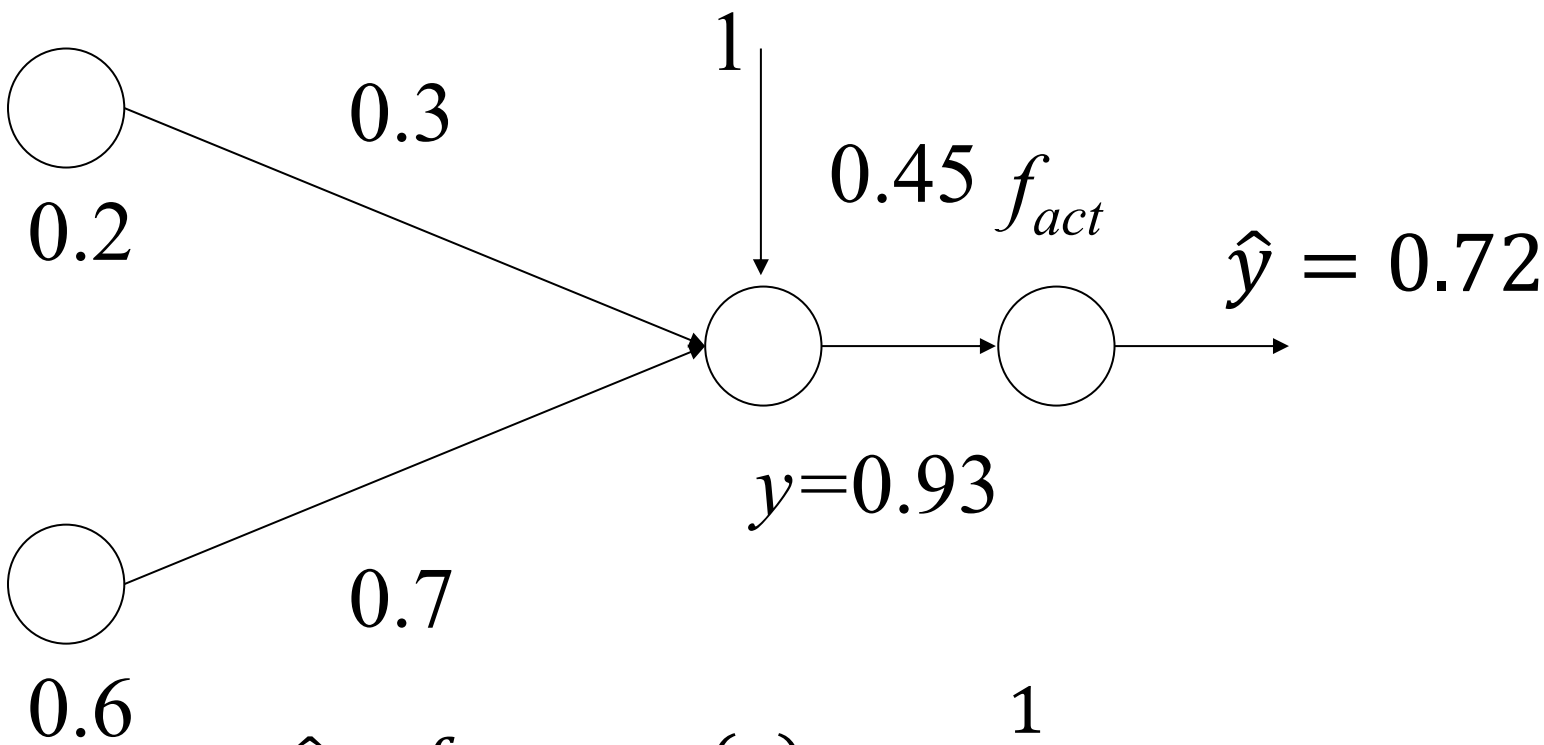
Source [2]

(C) Bipolar sigmoidal step

$$f_{act}(x) = \frac{1 - e^{-\lambda x}}{1 + e^{-\lambda x}}$$

Numerical Example 2

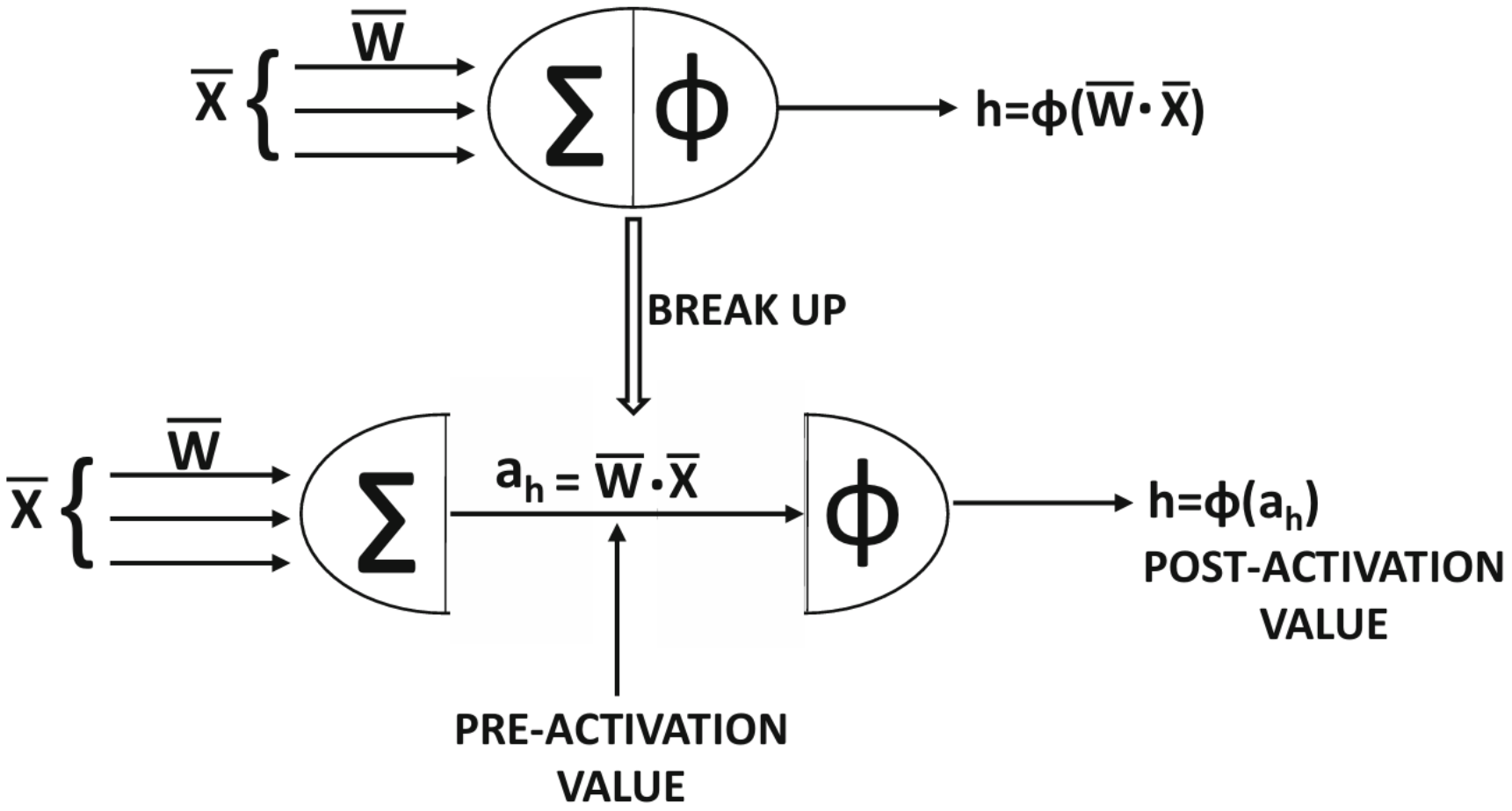
Calculate the output assuming binary sigmoidal activation function



$$\hat{y} = f_{activation}(y) = \frac{1}{1 + e^{-\lambda y}}$$

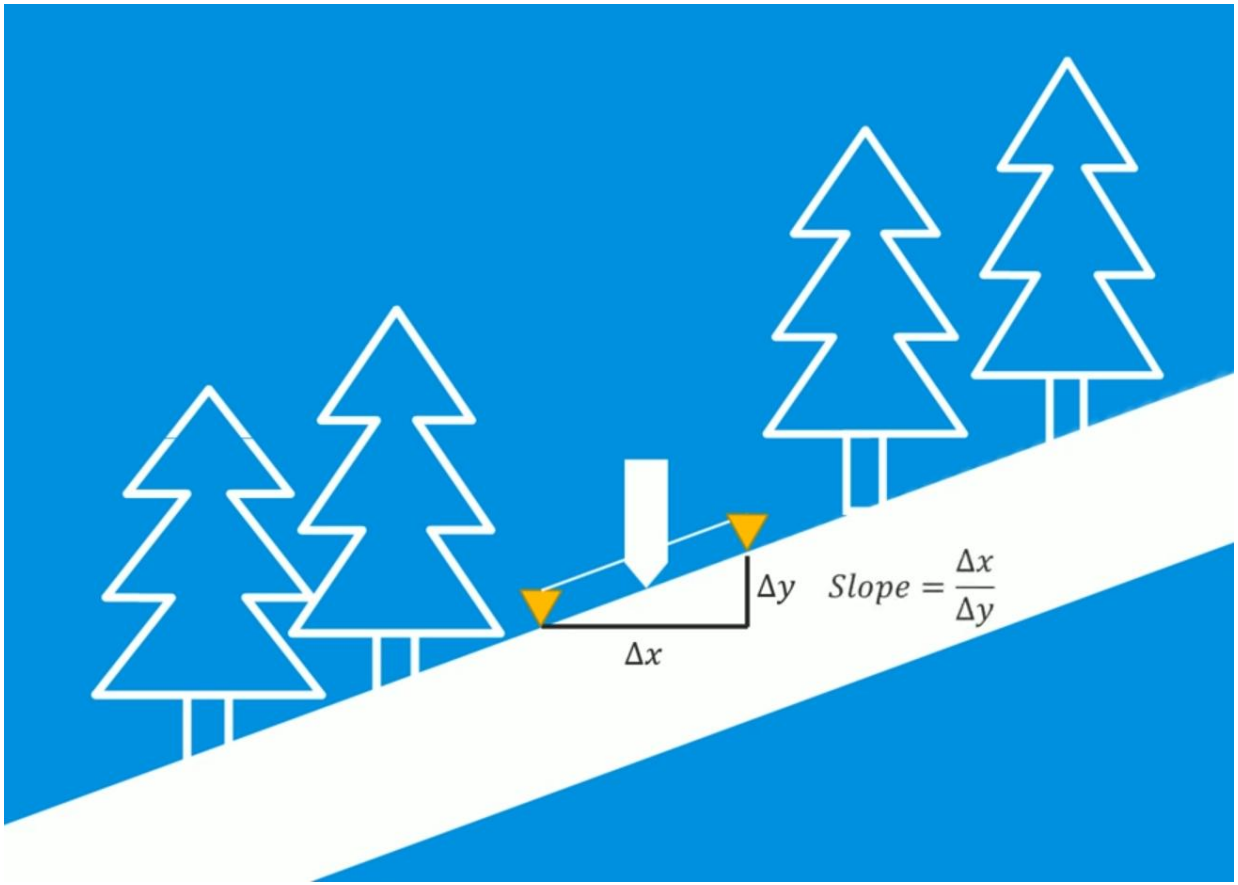
λ=1 is Binary sigmoidal function

Pre- and Post-Activation Values



Gradient Descent

- Gradient represents how STEEP a slope is.



Gradient Descent

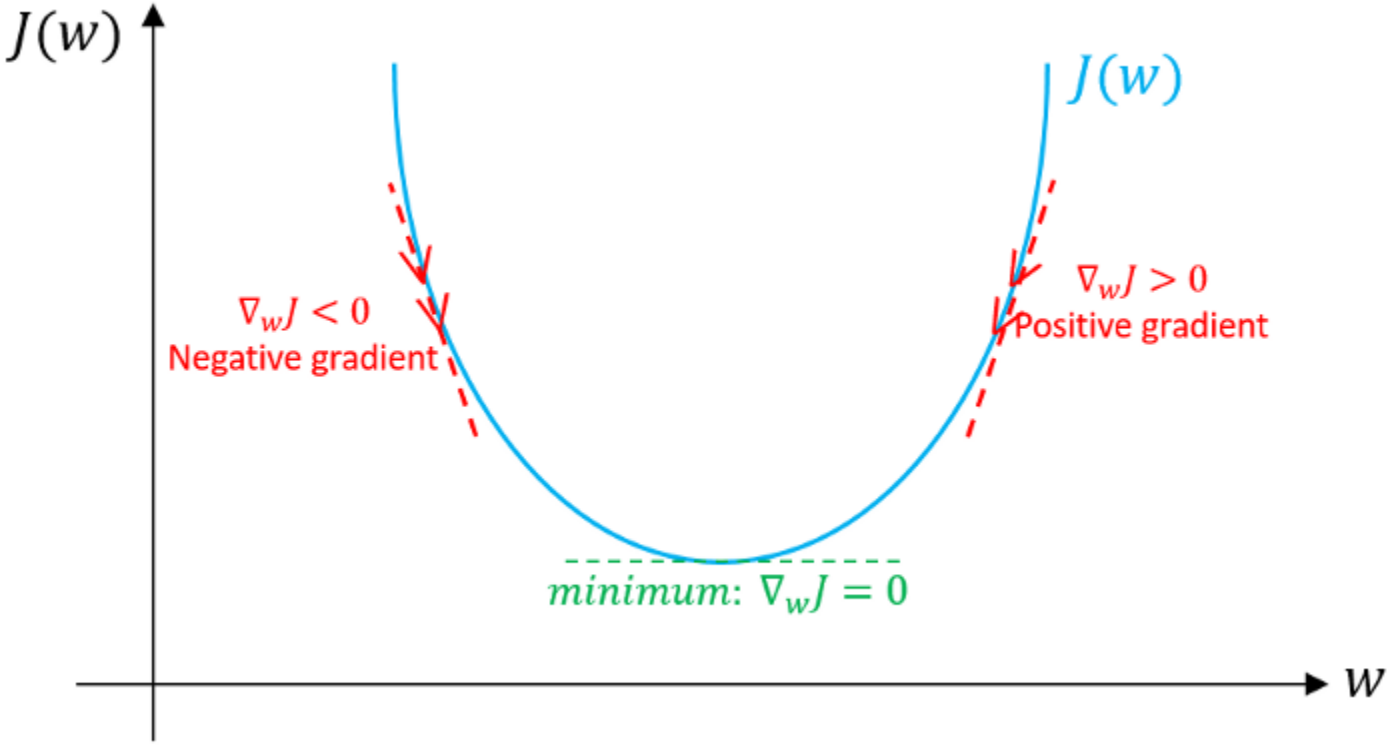
Given a differentiable function f(x),

gradient descent finds the minimum

by updating the variable x in steps proportional to the negative of the gradient (derivative) of f'(x) at the current point.

Gradient Descent

- Gradient represents how STEEP a slope is.
- Uphill is positive; downhill is negative.
- One direction – Derivative. multi-direction – Gradient.
- Drawback: Settles to local minimum



Gradient Search

For unconstrained optimization i.e., no constraints

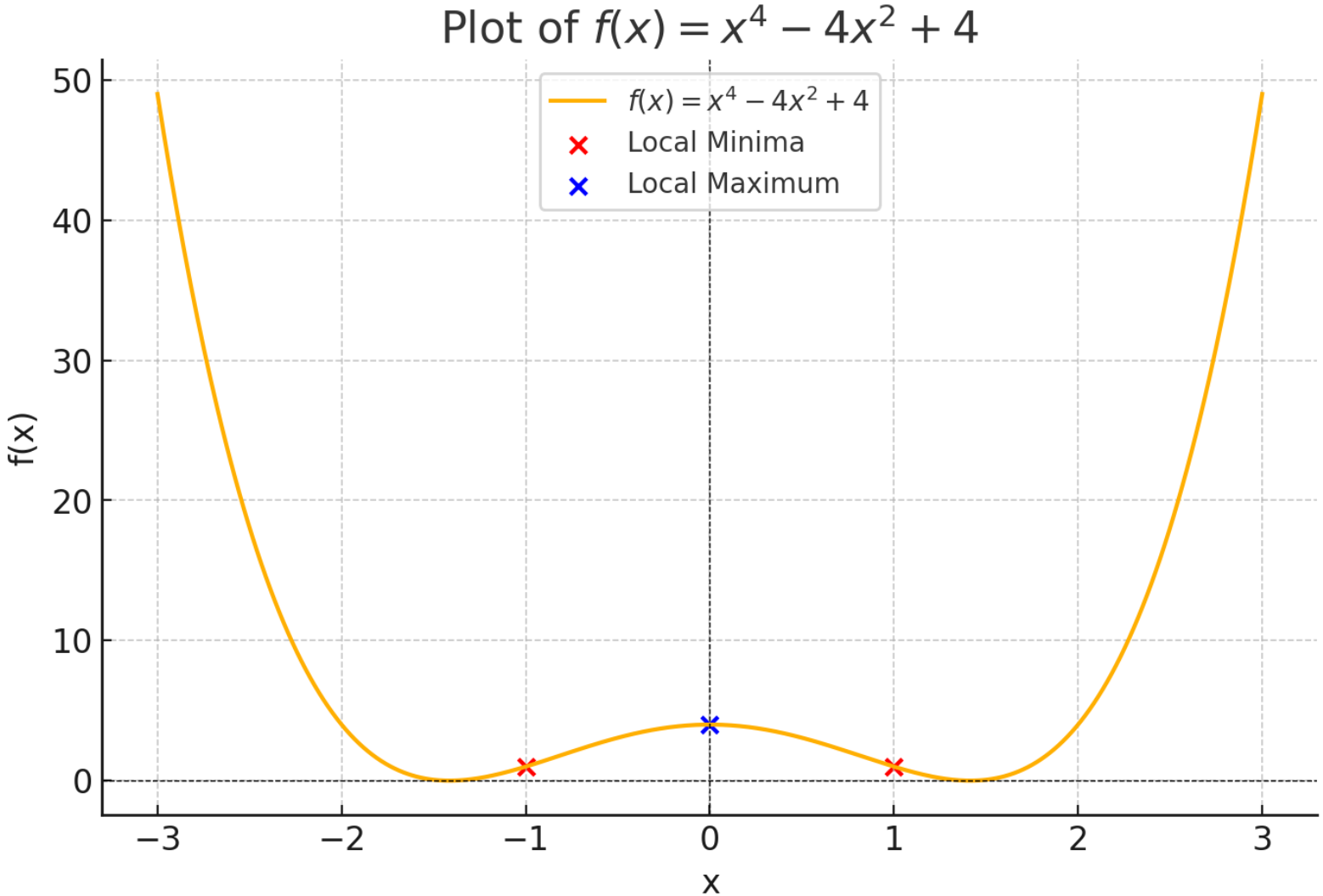
$$\underset{x}{\text{minimize}}\, f(x)$$

Gradient of function $f'(x)$ exists

1. Initialization. Choose an initial x^k and let $k = 0$
2. Calculate the derivative $f'(x^k)$, which indicates the slope of $f(x)$ at x^k
3. Update $x^{k+1} = x^k - \alpha f'(x^k)$. Where α is learning rate
4. Repeat steps 2 -3 till convergence i.e. $f'(x^k) \approx 0$, or the maximum number of iterations is reached.

$$\min_{x} x^4 - 4x^2 + 4$$

Gradient function $f'(x) = 4x^3 - 8x$



Case 1: Initial value x0=−2

Iteration (n)	x_n	f'(x_n)	x_n+1=x_n−αf′(x_n)	f(x_n)
0	−2	f′(−2)=−16	−2+0.1×16=−0.4-	f(−2)=12
1	−0.4	f′(−0.4)=1.28	−0.4−0.1×1.28=−0.528	f(−0.4)=3.23
2	−0.528	f′(−0.528)=1.39	−0.528−0.1×1.39=−0.667	f(−0.528)=2.9
3	−0.667	f′(−0.667)=1.48	−0.667−0.1×1.48=−0.815	f(−0.667)=2.65
4	−0.815	f′(−0.815)=1.6	−0.815−0.1×1.6=−0.975	f(−0.815)=2.5

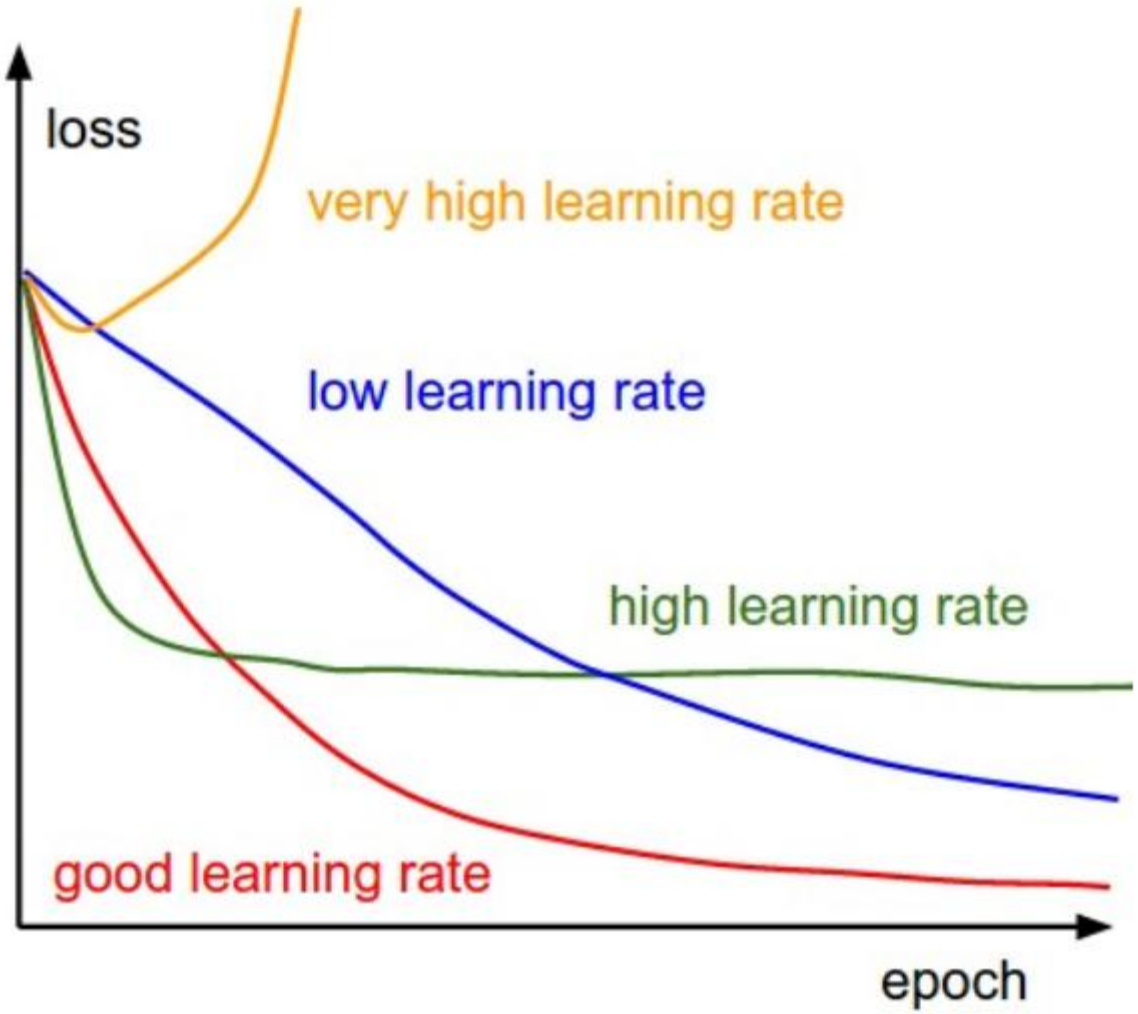
The trajectory is converging toward the local minimum at $x=-1$

Case 2: Initial value x0=2

Iteration (n)	x_n	f'(x_n)	x_n+1=x_n−αf′(x_n)	f(x_n)
0	2	f′(2)=16	2−0.1×16=0.42	f(2)=12
1	0.4	f′(0.4)=−1.28	0.4−0.1×−1.28=0.528	f(0.4)=3.23
2	0.528	f′(0.528)=−1.39	0.528−0.1×−1.39=0.667	f(0.528)=2.9
3	0.667	f′(0.667)=−1.48	0.667−0.1×−1.48=0.815	f(0.667)=2.65
4	0.815	f′(0.815)=−1.6	0.815−0.1×−1.6=0.975	f(0.815)=2.5

The trajectory is converging toward the local minimum at $x=1$

Effect of learning rate



Chain Rule

The chain rule is applied to calculate the gradients of the loss function with respect to the weights and biases (different layers) of the network. These gradients are then used to update the parameters during optimization (e.g., using gradient descent).

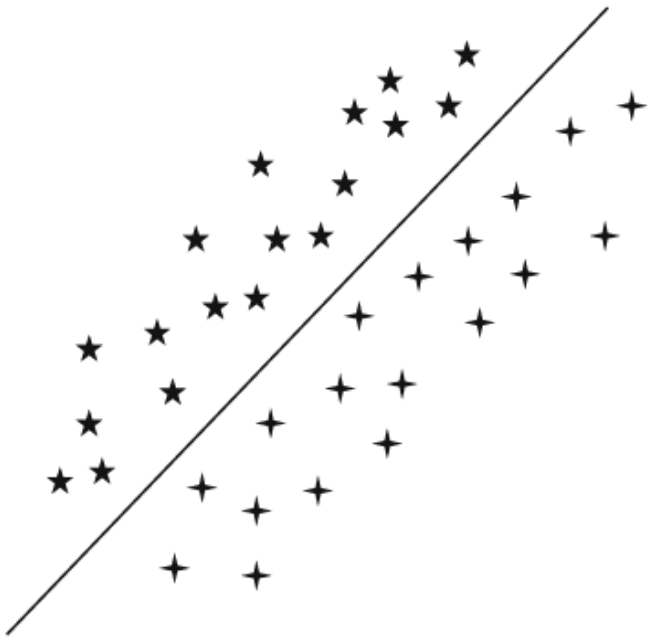
The Chain Rule: Formula

If you have a composite function $f\big(g(x)\big)$, the derivative with respect to x is given by:

$$\frac{d}{dx} f\big(g(x)\big) = f'\big(g(x)\big).\, g'(x)$$

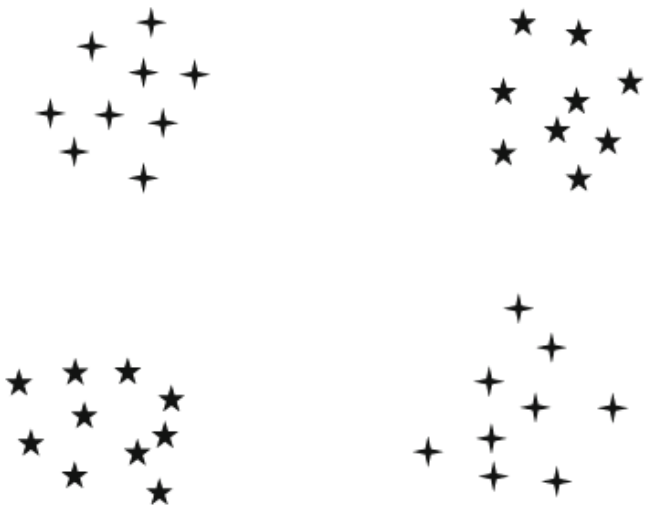
This principle is used extensively in updating the weights of different layers using the Loss function at the outside layer

Linear Separability



$\overline{W} \cdot \overline{X}^T = 0$

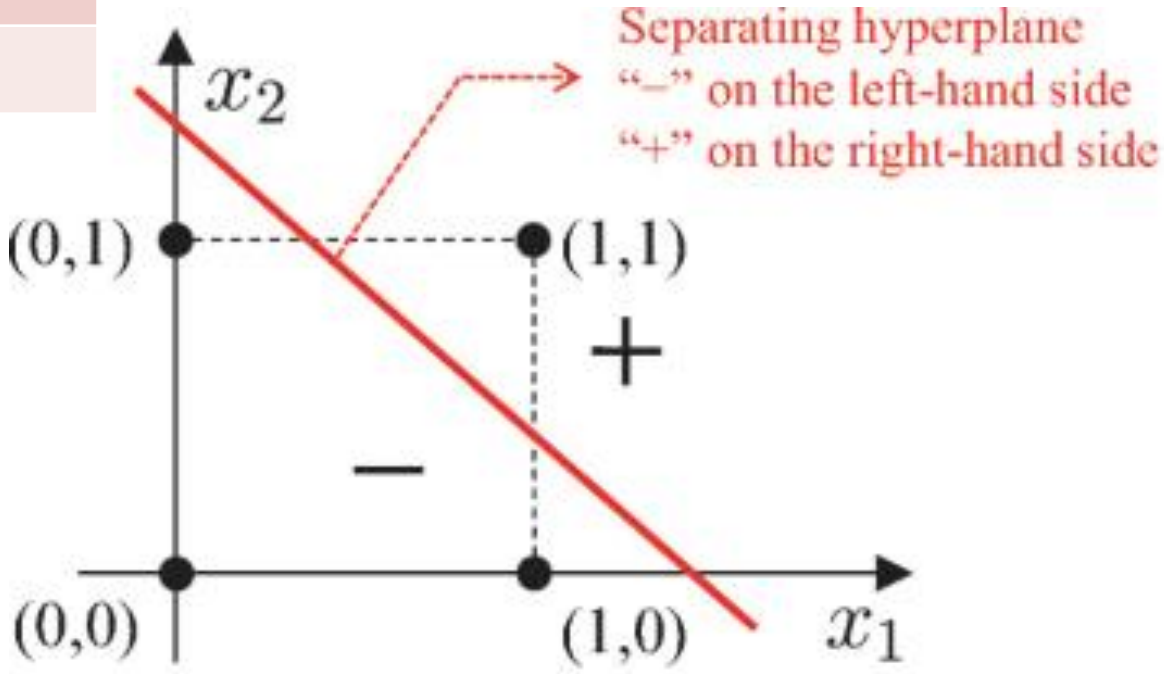
LINEARLY SEPARABLE



NOT LINEARLY SEPARABLE

Linearly Separable – AND gate

x_1 (input)	x_2 (input)	Y (output)
0	0	0
0	1	0
1	0	0
1	1	1

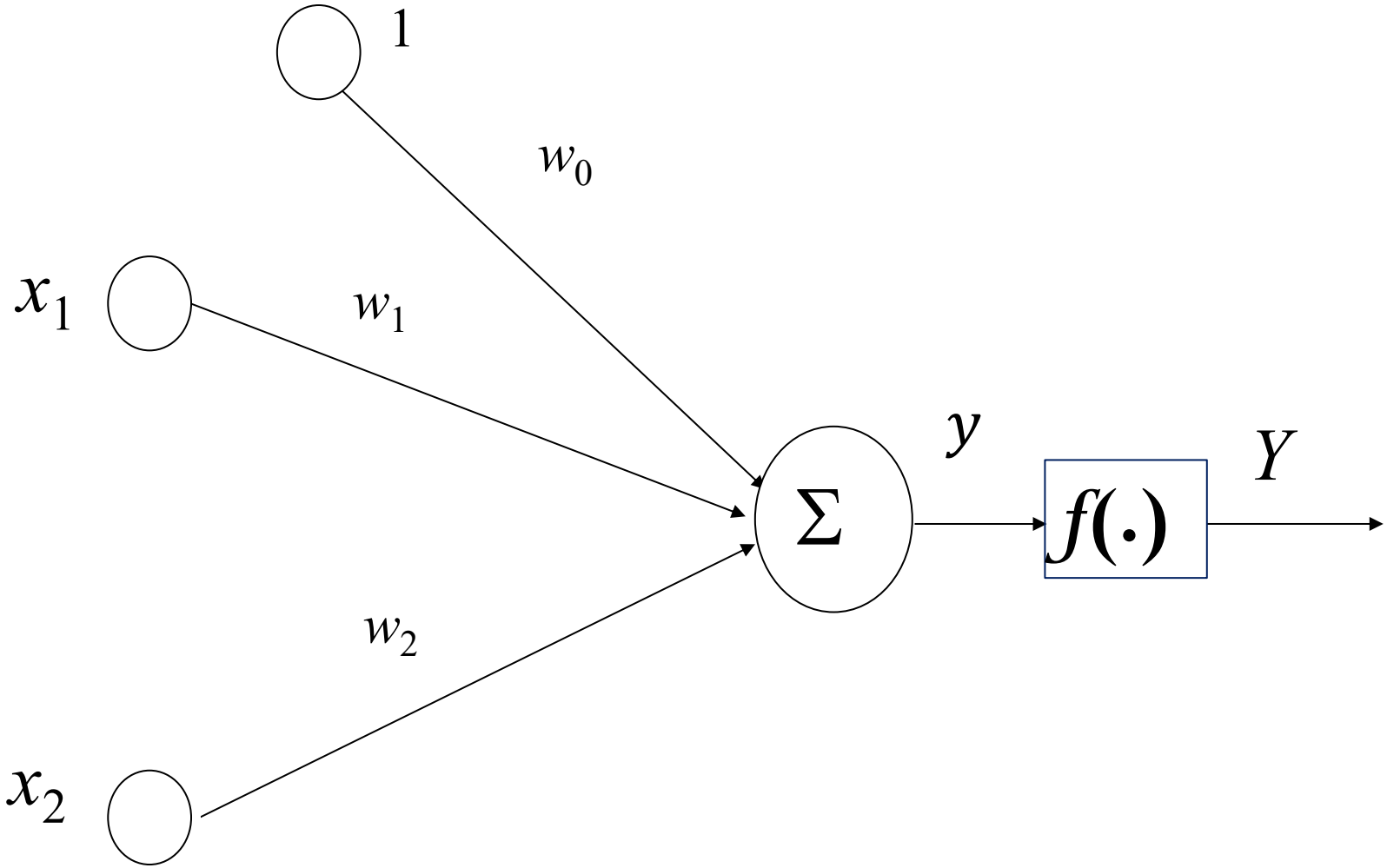


Linearly Separable – AND gate

- Two input sources => Two input neurons
- One output => One output Neuron.
- Activation function is binary sigmoidal $f(y) = \frac{1}{1+e^{-y}}$
- Derivative $f'(y) = \frac{-1}{(1+e^{-y})^2}(-e^{-y})$

$$= \frac{1}{1+e^{-y}}\left[1 - \frac{1}{1+e^{-y}}\right] = f(y)(1 - f(y))$$

Linearly Separable – AND gate



Back-propagation training/algorithm

Given: Input vector i^{th} instant $X_i = [x_1\ x_2]_i$, Target T_i.

Initialize weights w_0, w_1, w_2 and learning rate with some random values in the range [0 1]

1. Output $y = 1.w_0 + x_1.w_1 + x_2.w_2$
2. Activation function $Y = f(y)$ sigmoidal activation function
3. Compute error: $e = \frac{1}{2}(Y - T_i)^2$
4. Backpropagate the error to crossing activation function

$\delta = \frac{\partial e}{\partial Y} f'(y)$ where $f'(y)$ is the derivative of activation function selected.

$\delta = (Y - T_i) f(y)(1 - f(y))$ for sigmoidal activation function

Back-propagation training/algorithm

5. Compute change in weights and bias

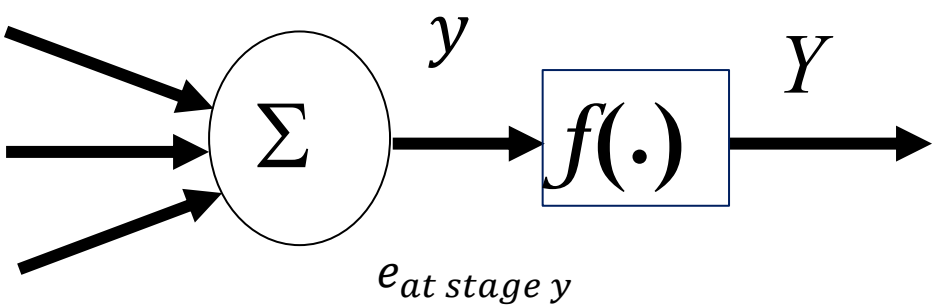
$$\Delta w_0 = \delta.1, \Delta w_1 = \delta x_1, \Delta w_2 = \delta x_2$$

6. Update the changes in weights and bias

$$w_0 = w_0 - \alpha \Delta w_0; w_1 = w_1 - \alpha \Delta w_1; w_2 = w_2 - \alpha \Delta w_2$$

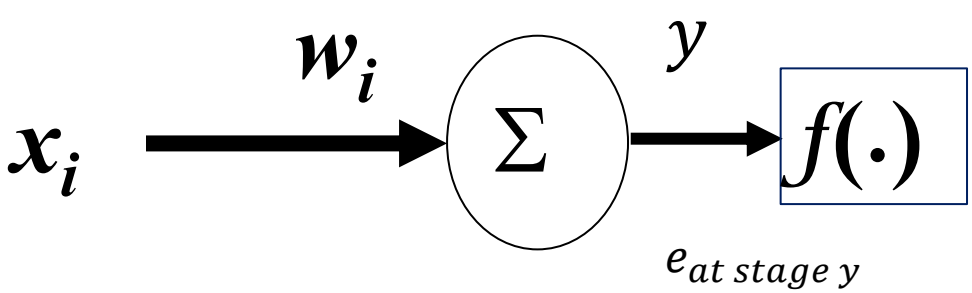
7. Keep repeating the steps 1 – 6, for all input combinations (4 nos). This is one epoch.
8. Run multiple Epochs till the error decreases and stabilizes.

(4 Rules)Backpropagating Error



1. Output Neuron

$$e_{at\ stage\ y} = \frac{\partial Loss\ function}{\partial Y} f'(y)$$



2. Across Link

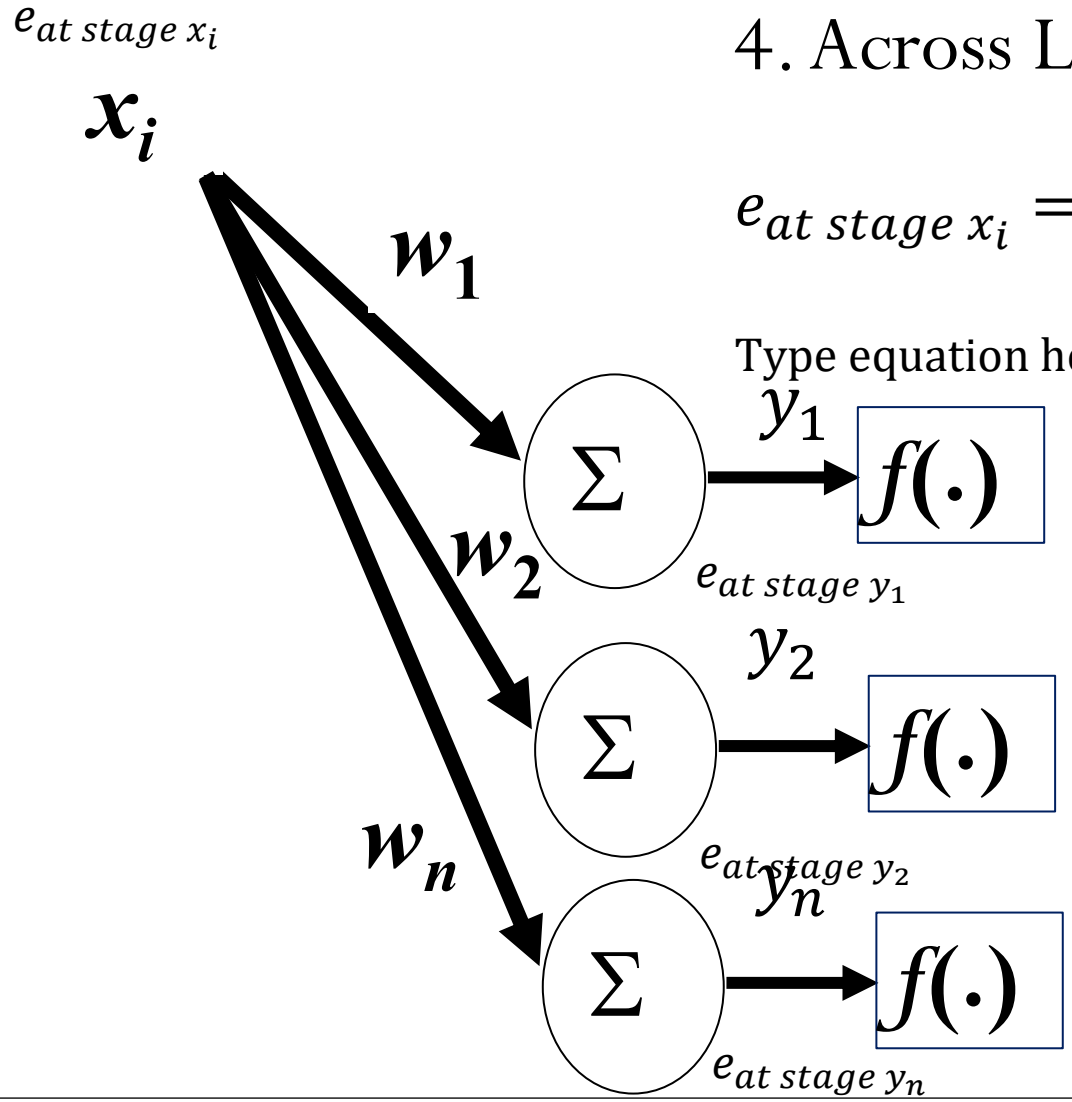
$$e_{at\ stage\ x_i} = w_i \times e_{at\ stage\ y}$$

$e_{at\ stage\ x_i}$

3. Weights Update

$$\Delta w_i = w_i \times e_{at\ stage\ y};\ w_i = w_i - \alpha \Delta w_i$$

(4 Rules)Backpropagating Error

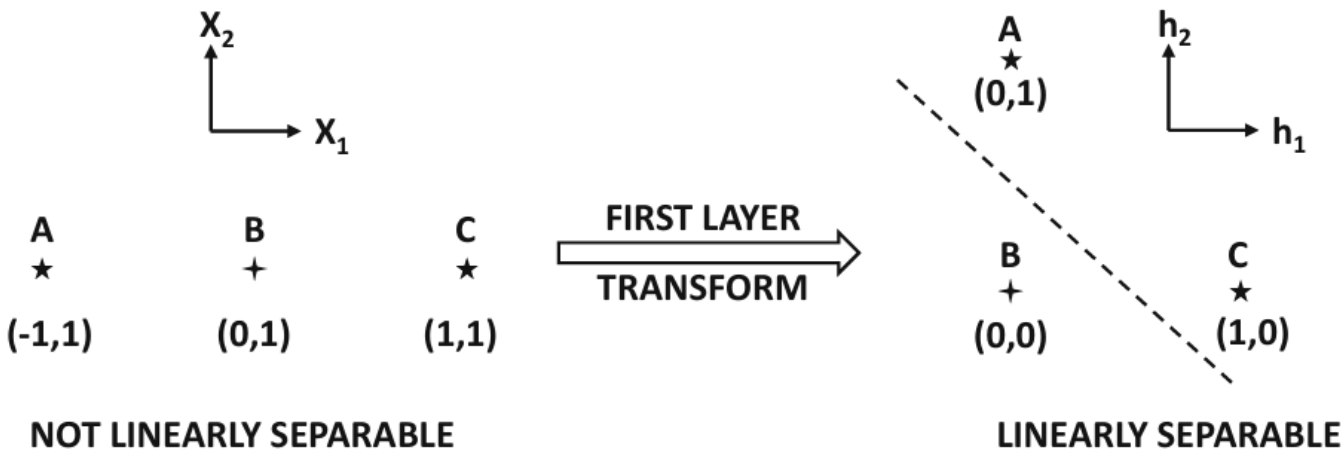


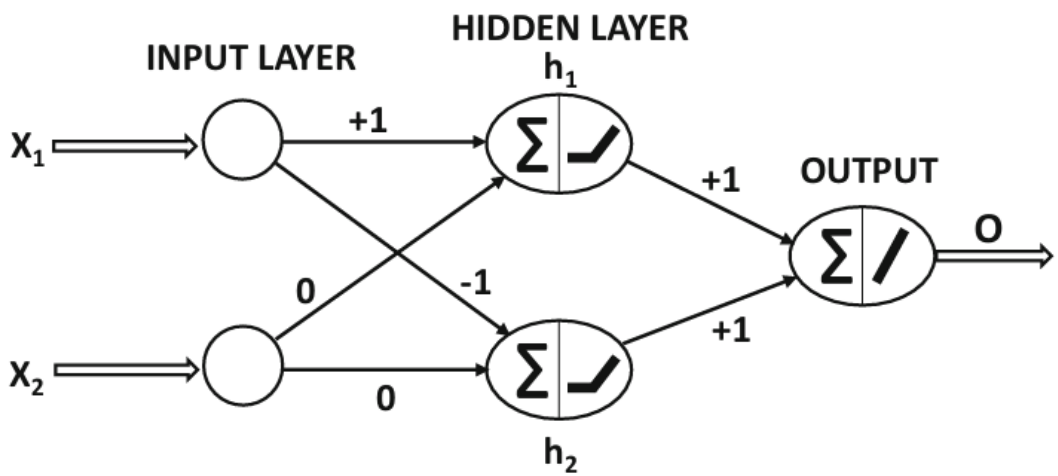
4. Across Link (>*1 hidden layer*)

$$e_{at\ stage\ x_i} = \sum_{i=1}^{n} w_i \times e_{at\ stage\ y_i}$$

Type equation here.

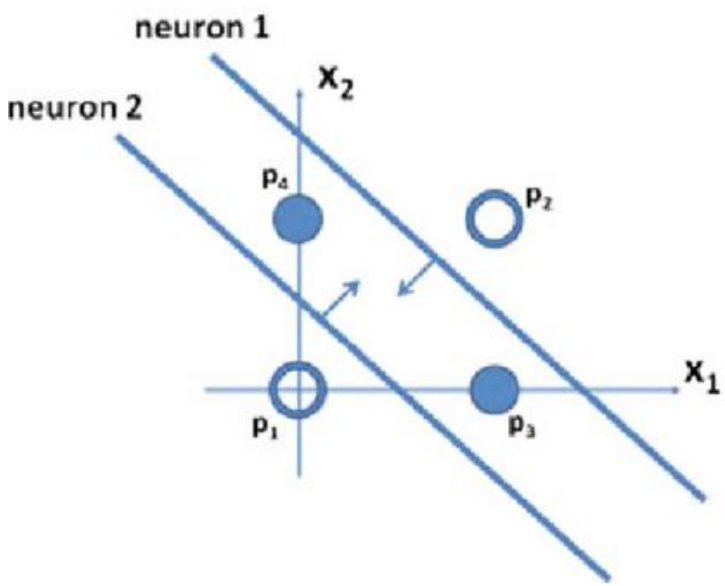
The power of nonlinear activation functions in transforming a data set to linear separability

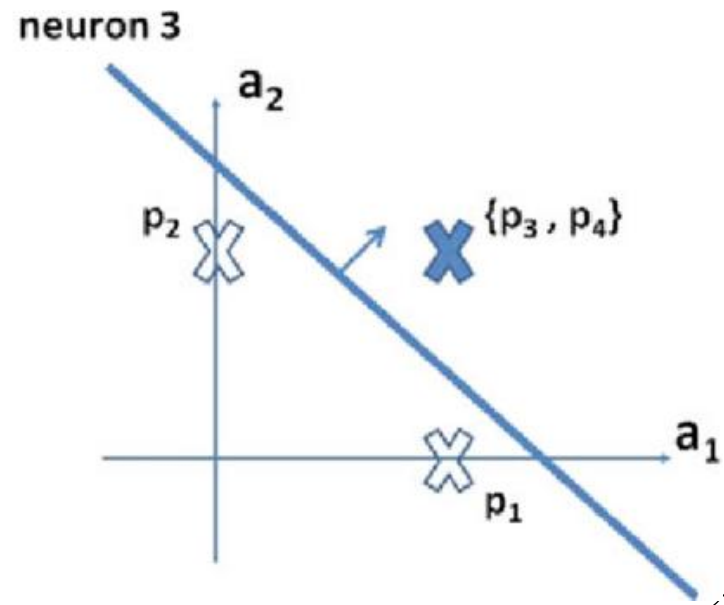




Linearly not Separable – XOR gate

x_1 (input)	x_2 (input)	Y (output)
0	0	0
0	1	1
1	0	1
1	1	0





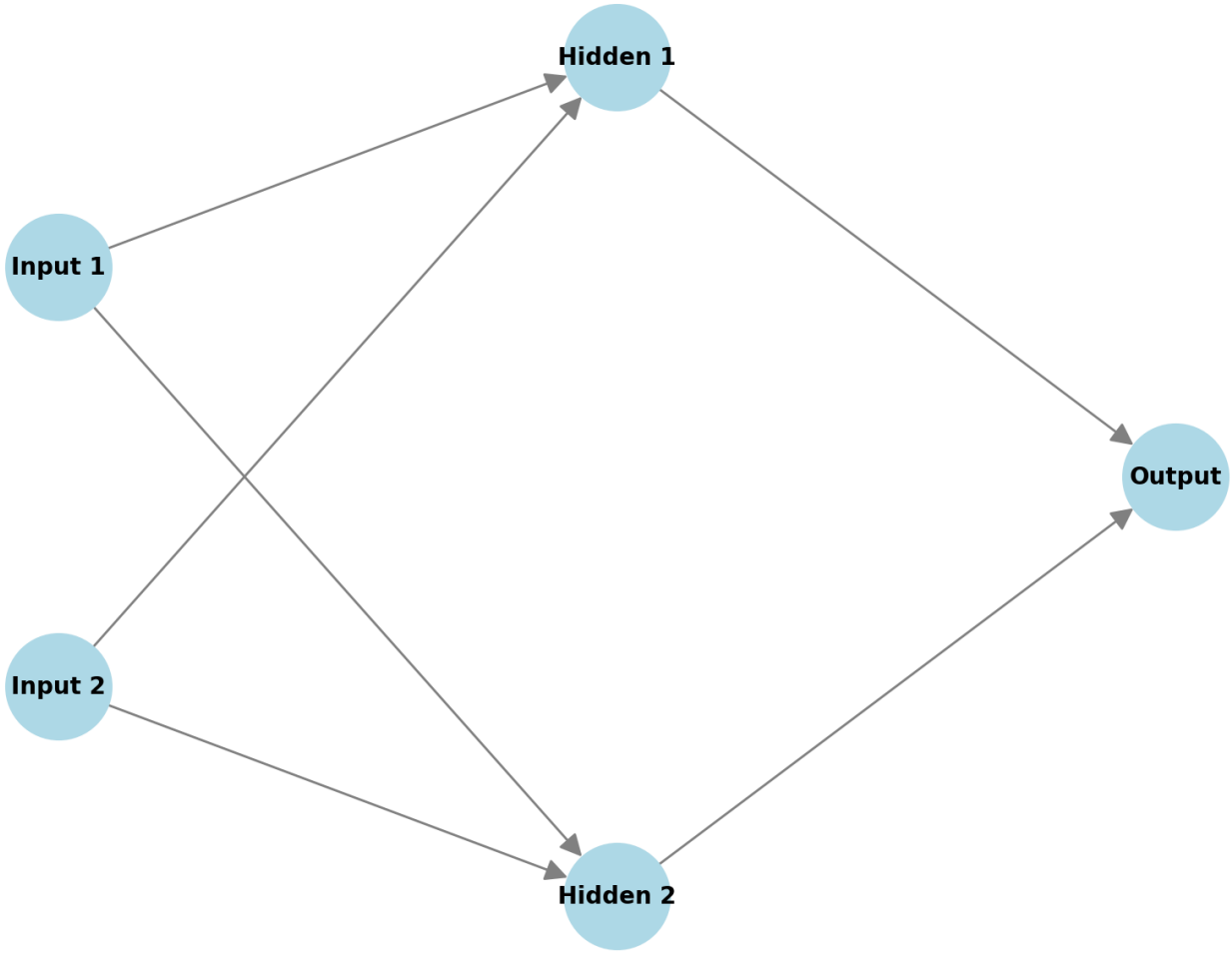
Linearly not Separable – XOR gate

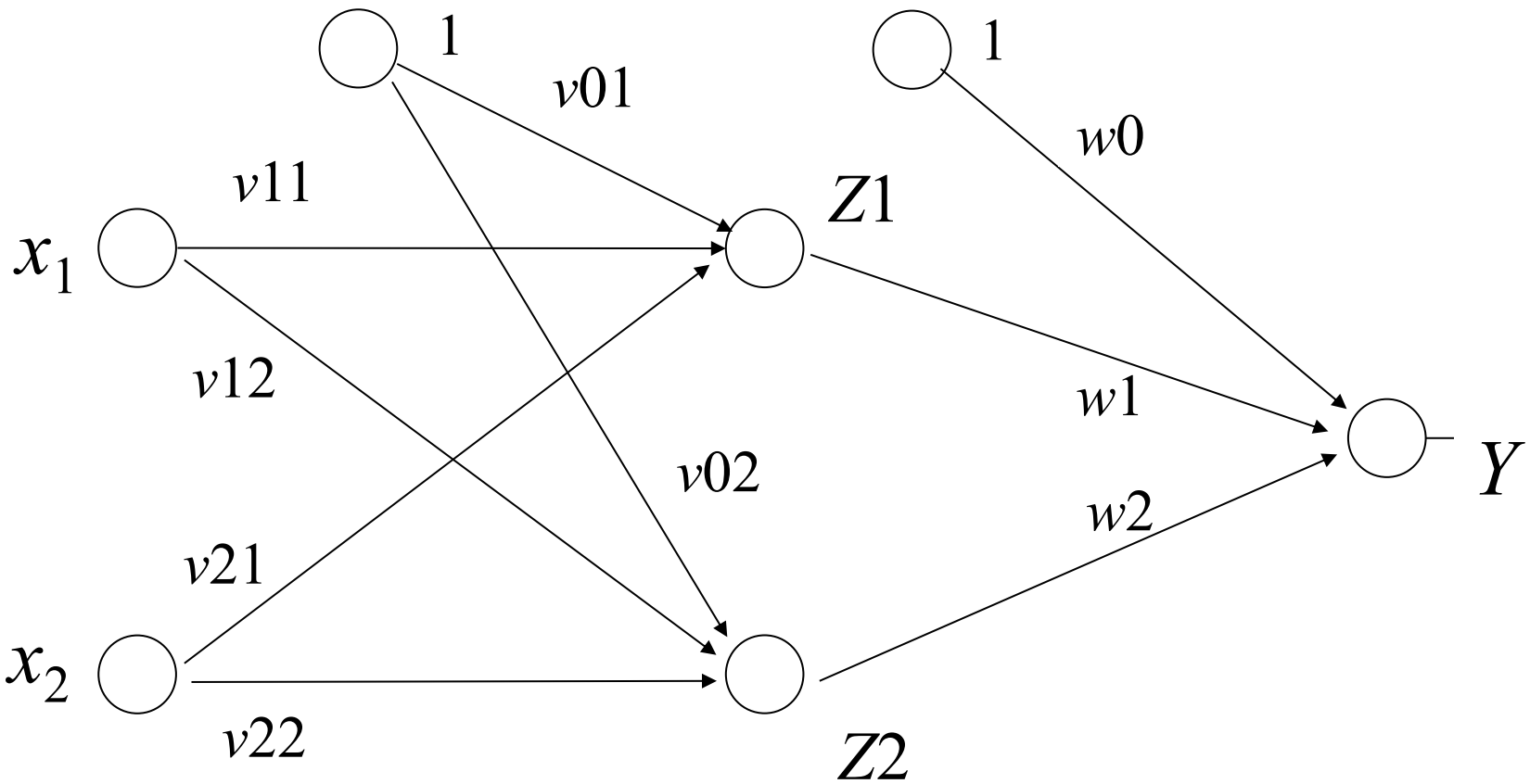
- Two input sources => Two input neurons
- One output => One output Neuron.
- One hidden layer => 2 neurons
- Activation function is binary sigmoidal $f(y) = \frac{1}{1+e^{-y}}$
- Derivative $f'(y) = \frac{-1}{(1+e^{-y})^2}(-e^{-y})$

$$= \frac{1}{1+e^{-y}}\left[1 - \frac{1}{1+e^{-y}}\right] = f(y)(1 - f(y))$$

Linearly not Separable – XOR gate

Neural Network for XOR Gate




Input layer
Hidden layer
Output layer
1
1
v01
w0
v11
Z1
x1
v12
w1
v02
Y
w2
v21
x2
v22
Z2

Back-propagation Training

Given: Inputs X_i, target t_i.

Feed-forward Phase

Initialize weights and learning rate with some random values

1. Hidden unit $z_j = \sum_{i=1}^{n} v_{i,j}\, x_i + v_{0,j}$, $j = 1$ to p hidden neurons
2. output $Z_j = f(z_j)$, sigmoidal activation function
3. Output unit $y_k = \sum_{j=1}^{p} w_{j,k}\, Z_j + w_{o,k}$
4. Output $Y = f(y_k)$ sigmoidal activation function

Back-propagation Training

Back-propagation of error Phase

5. Compute error correction term

   $\delta_k = (t_k - Y_k) f'(y_k)$ where $f'(y_k)$ is derivative

6. Compute change in weights and bias

   $\Delta w_{j,k} = \delta_k Z_j, \Delta w_{0,k} = \delta_k$

   send δ_k to previous layer

5. Hidden unit $\delta_{j,k} = \sum_{k=1}^{m} \delta_k w_{j,k}$

6. Calculate error term $\delta_j = \delta_{j,k} f'(z_j)$

7. Compute change in weights and bias

   $\Delta v_{i,j} = \delta_j x_i, \Delta v_{0,j} = \delta_j$

Back-propagation Training

Weights and Bias update phase

10. Each output unit, $k = 1$ to m update weights and bias

$$w_{j,k}^{new} = w_{j,k}^{old} - \alpha \Delta w_{j,k}$$
$$w_{0,k}^{new} = w_{0,k}^{old} - \alpha \Delta w_{0,k}$$

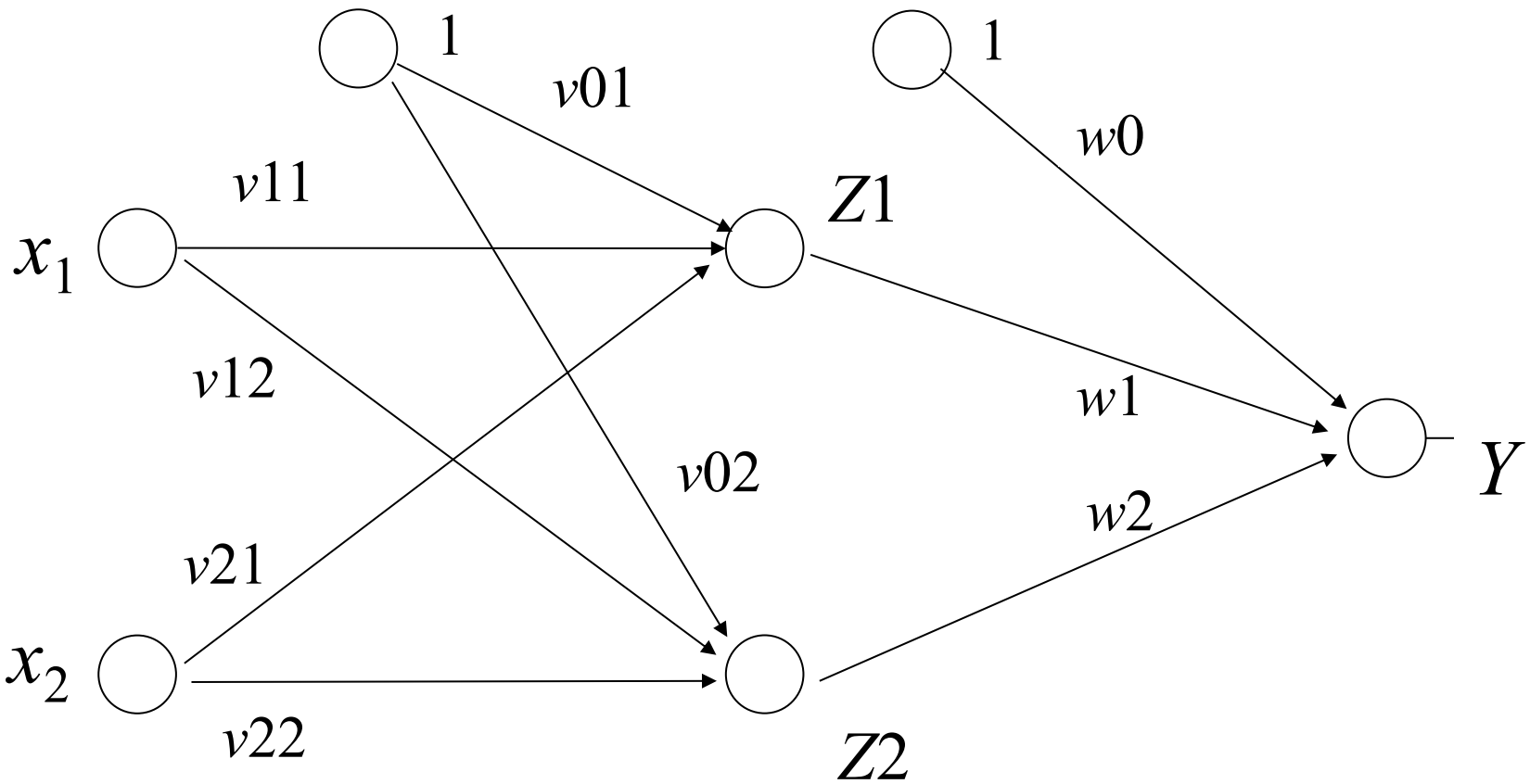
11. Each hidden unit, $j = 1$ to p update weights and bias

$$v_{i,j}^{new} = v_{i,j}^{old} - \alpha \Delta v_{i,j}$$
$$v_{0,j}^{new} = v_{0,j}^{old} - \alpha \Delta v_{0,j}$$

12. Check for stopping criterion e.g. certain number of epochs or when targets are equal/close to network outputs

Hidden neuron input computation

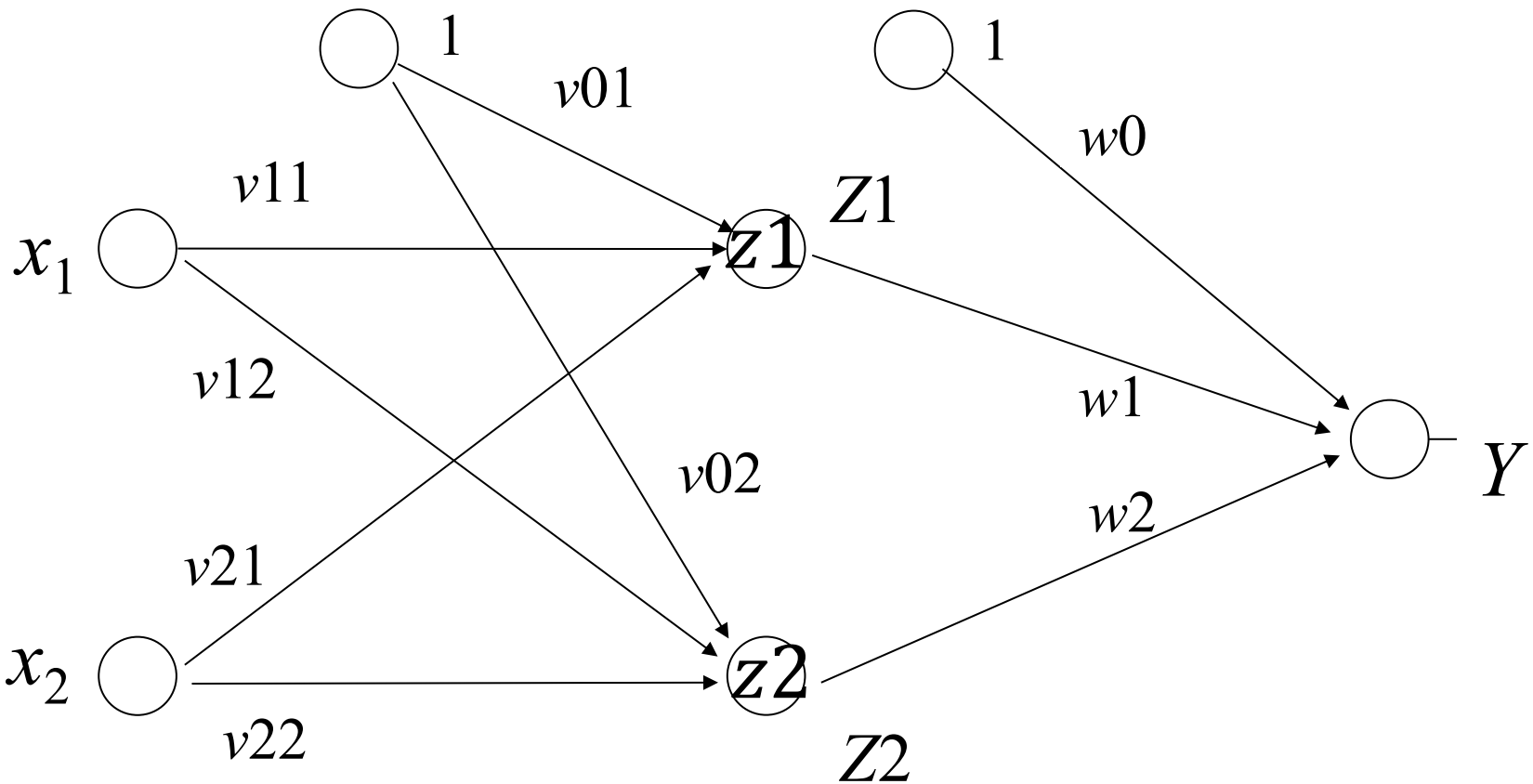
$$z_1 = \mathrm{v11}\, x_1 + \mathrm{v21}\, x_2 + v01$$



$$z_2 = \mathrm{v12}\, x_1 + \mathrm{v22}\, x_2 + v02$$

Hidden neuron output computation

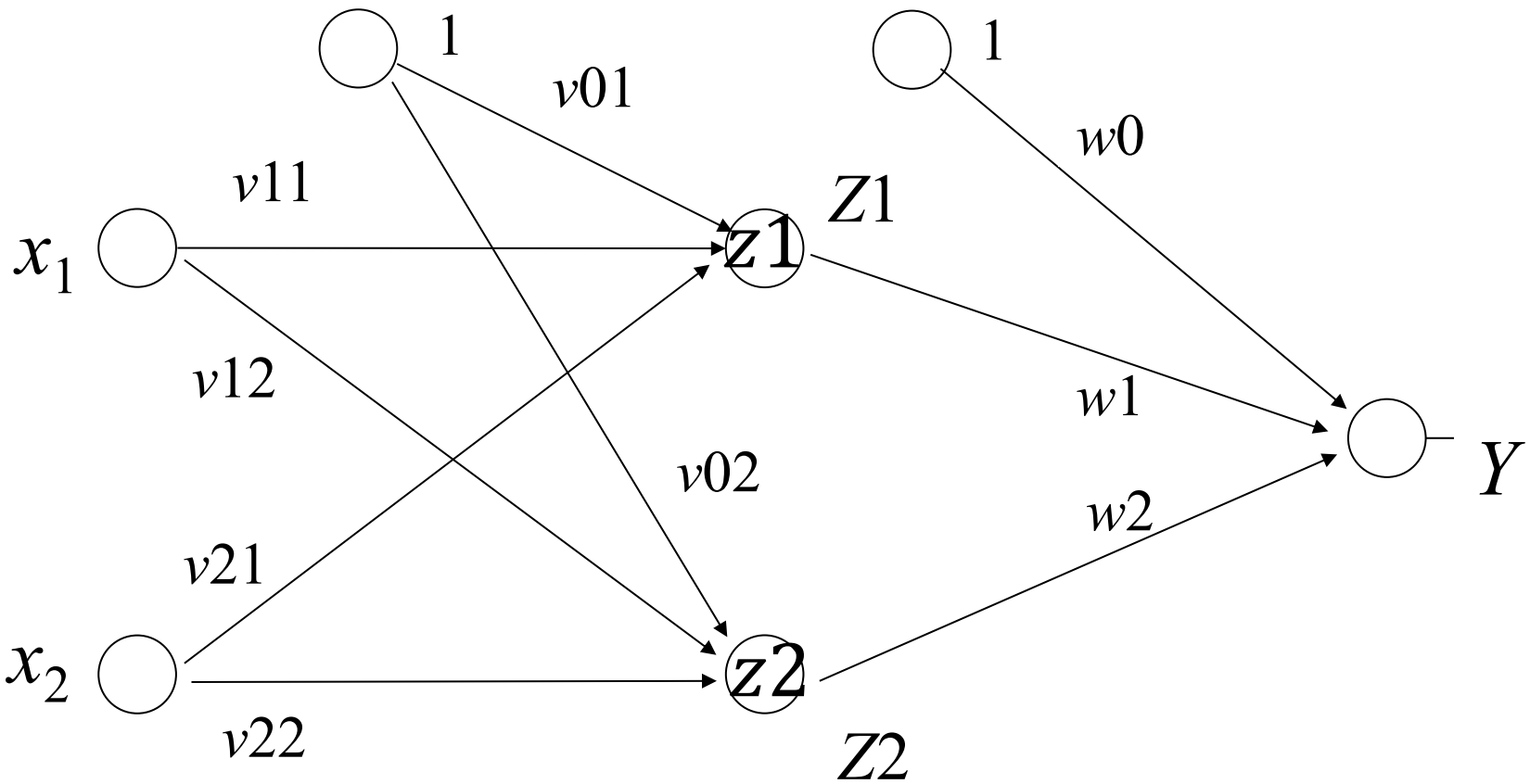
$$Z1 = f_{sig}(z1)$$



$$Z2 = f_{sig}(z2)$$

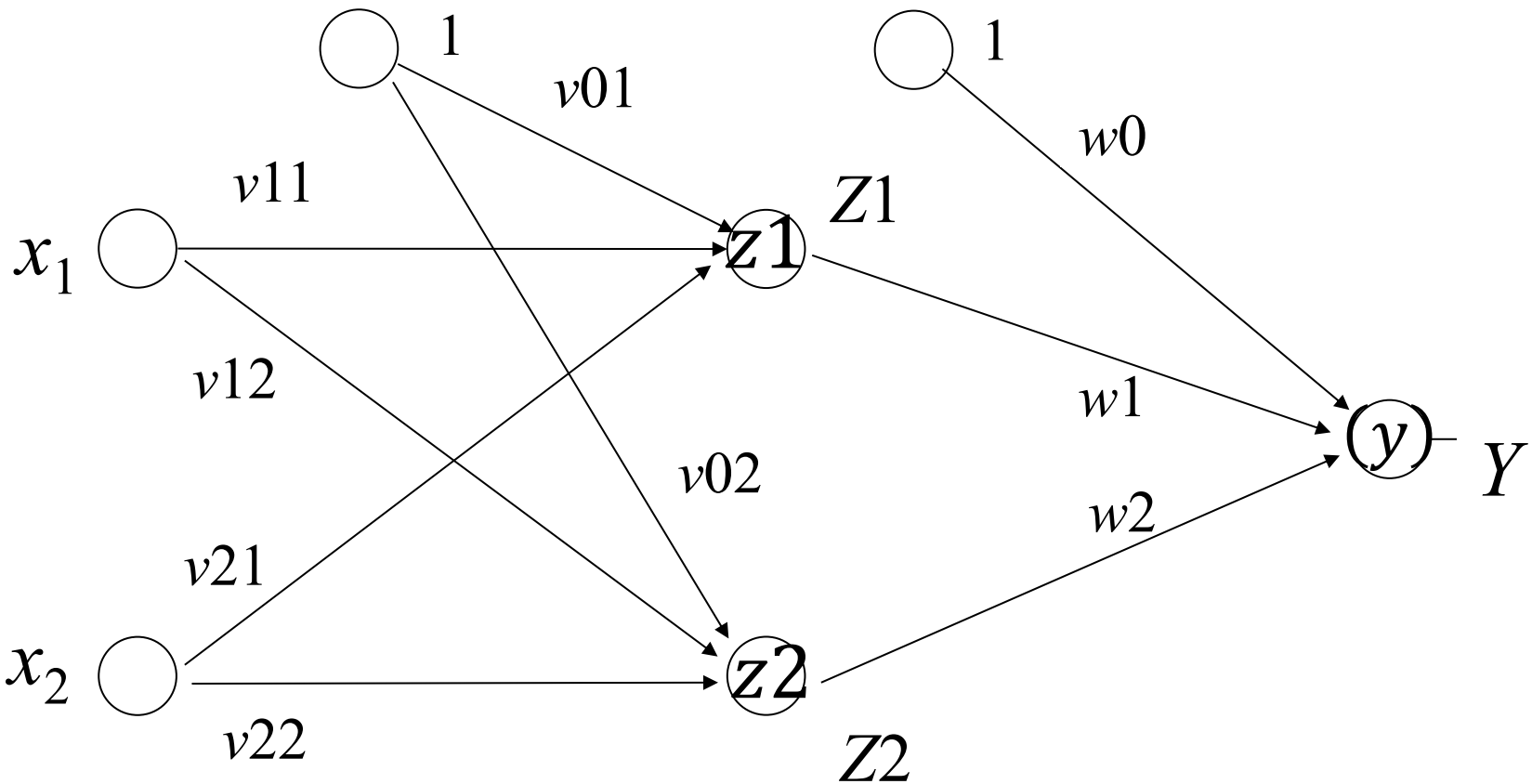
Output neuron input computation

$$y = w1Z1 + w2Z2 + w0$$



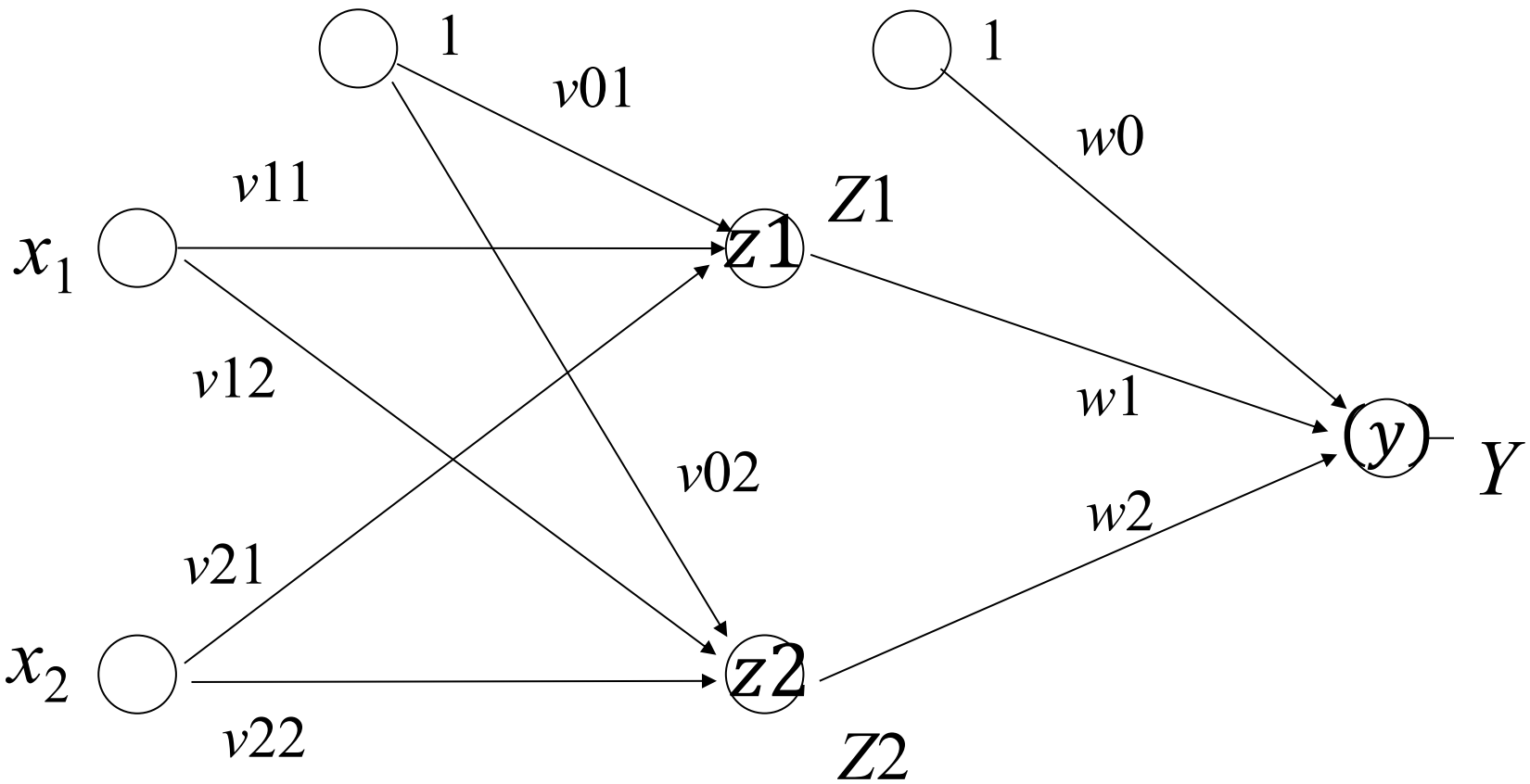
Output neuron Output computation

$$Y = f_{sig}(y)$$



Output Error correction computation

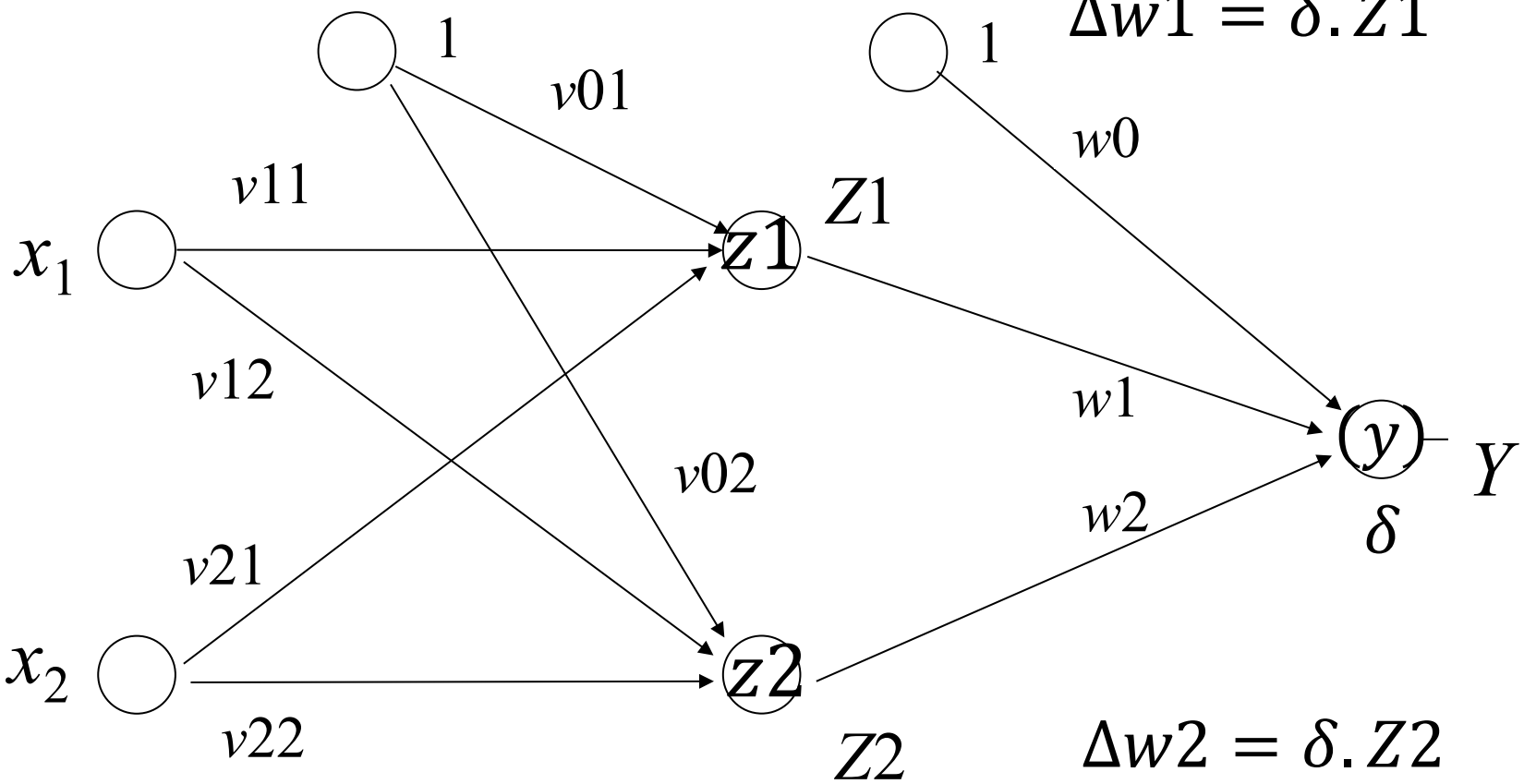
$$\delta = (t - Y)f'(y)$$



Output neuron changes updates computation

$$\Delta w0 = 1.\delta$$

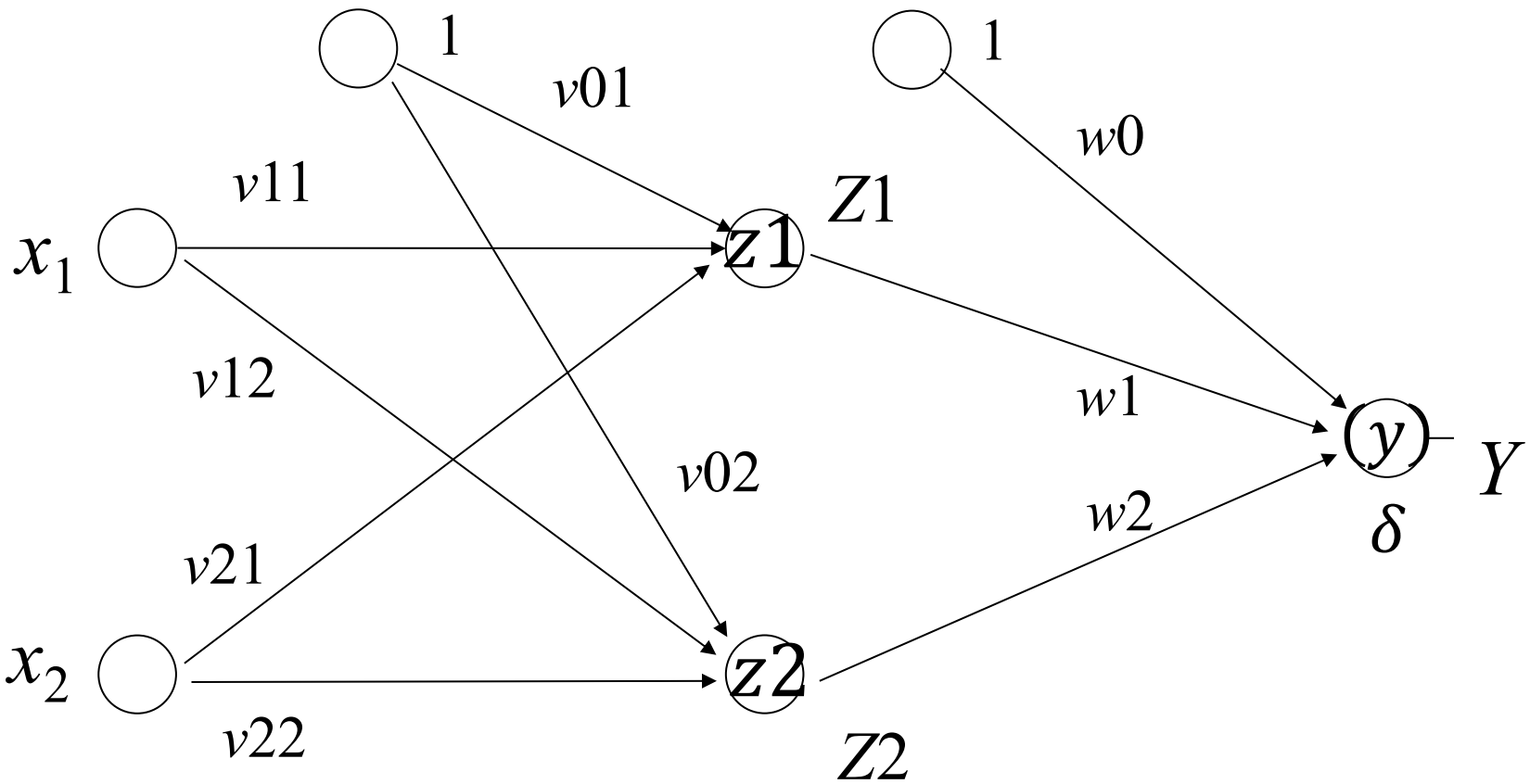
$$\Delta w1 = \delta.Z1$$



$$\Delta w2 = \delta.Z2$$

Hidden neuron error propagation computation

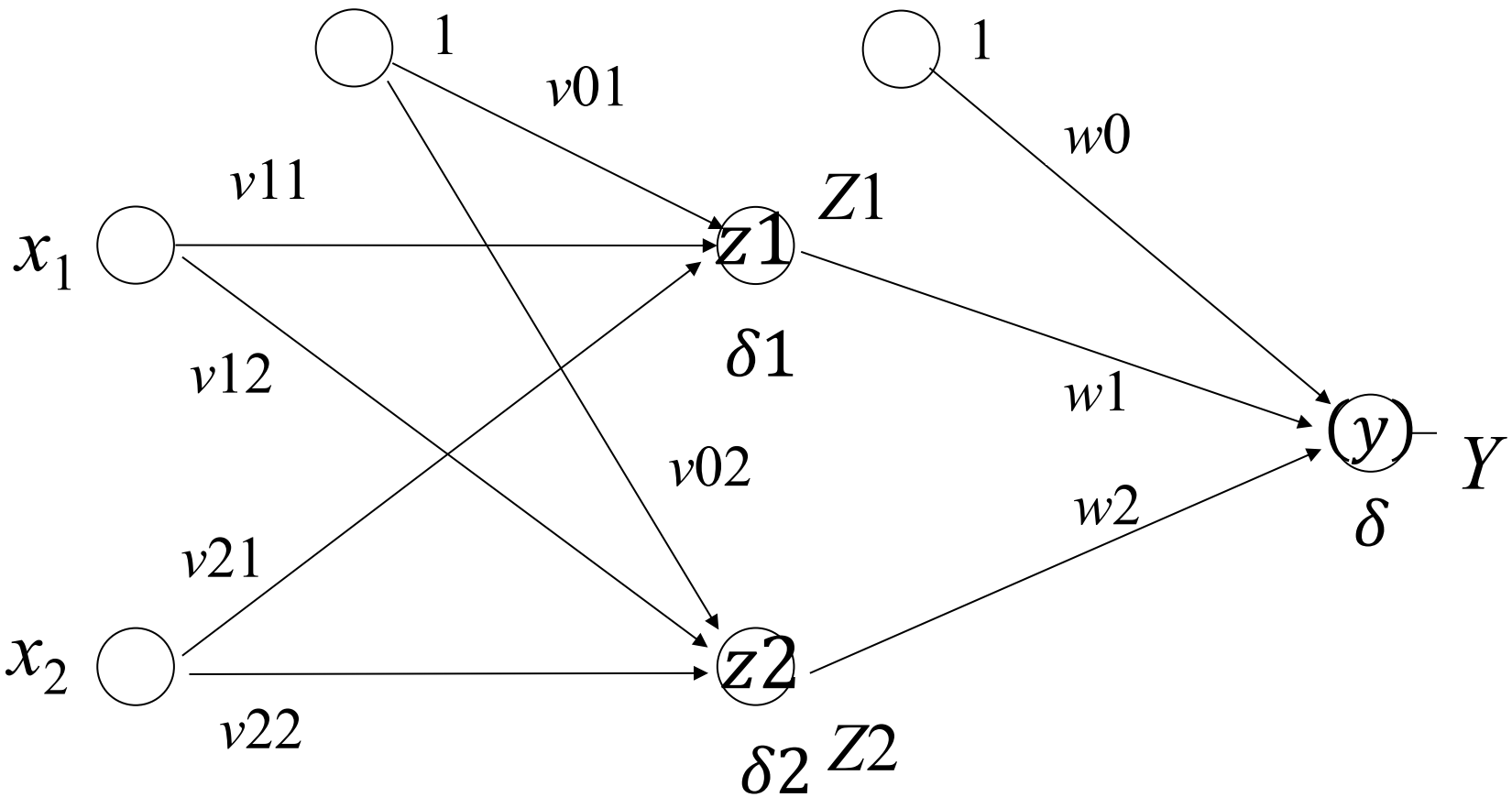
$$\delta 1 = \delta . w1$$



$$\delta 2 = \delta . w2$$

Hidden neuron error correction computation

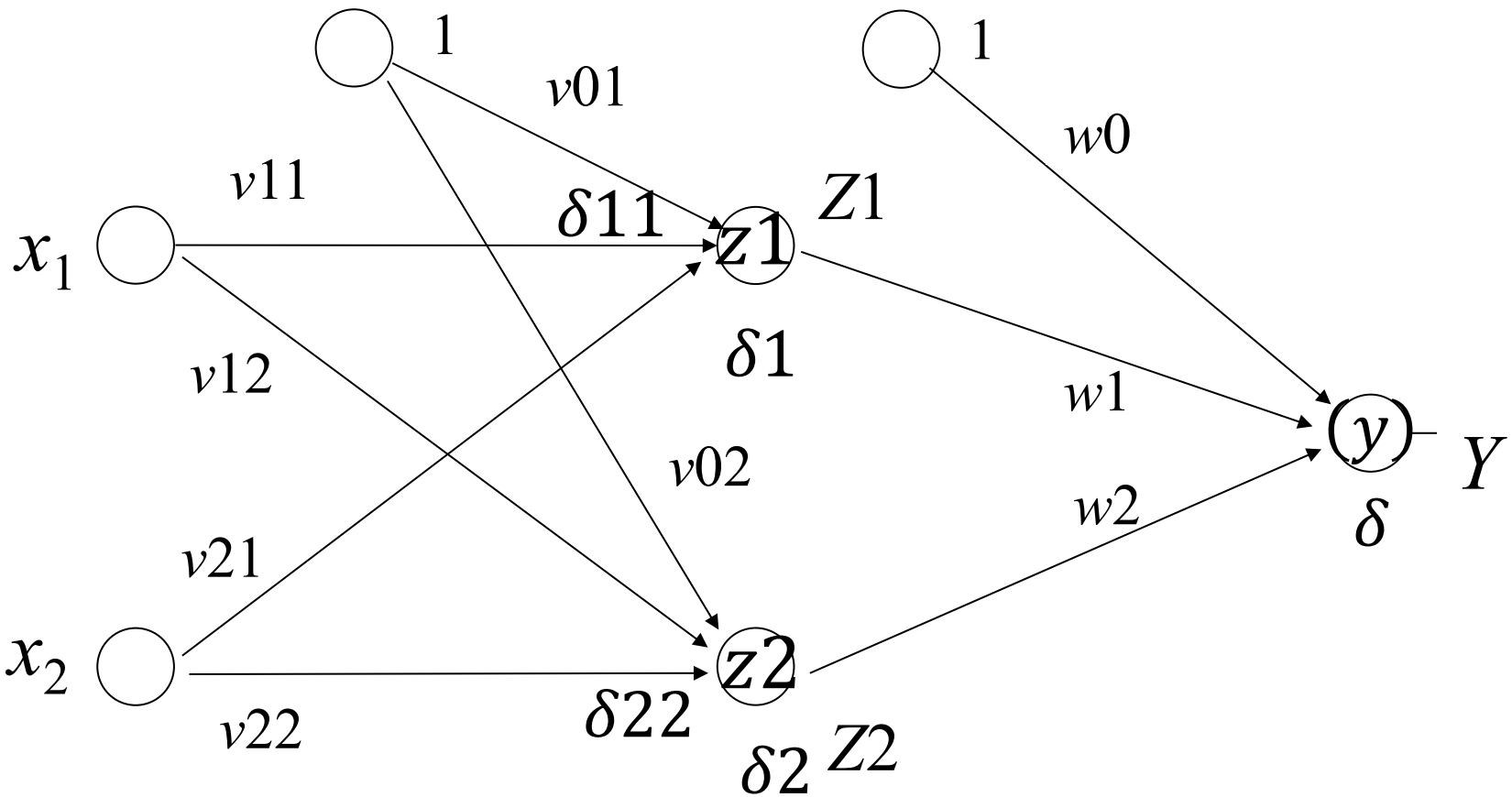
$$\delta 11 = \delta 1.f'(z1)$$



$$\delta 22 = \delta 2.f'(z2)$$

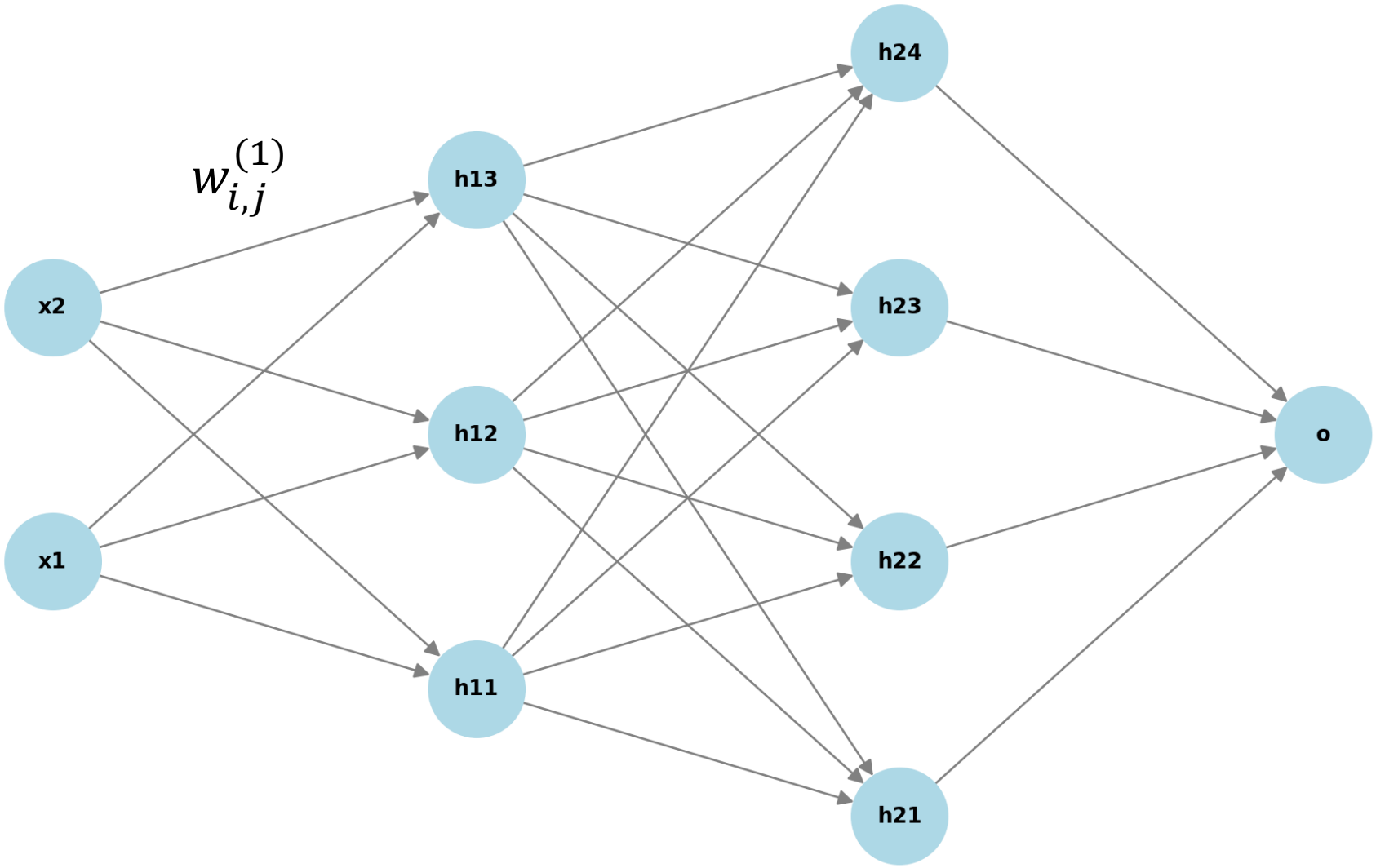
Hidden neuron changes updates computation

$\Delta v01 = \delta 11$ $\Delta v11 = \delta 11 x_1$ $\Delta v21 = \delta 11 x_2$



$\Delta v02 = \delta 22$ $\Delta v12 = \delta 22 x_1$ $\Delta v22 = \delta 22 x_2$

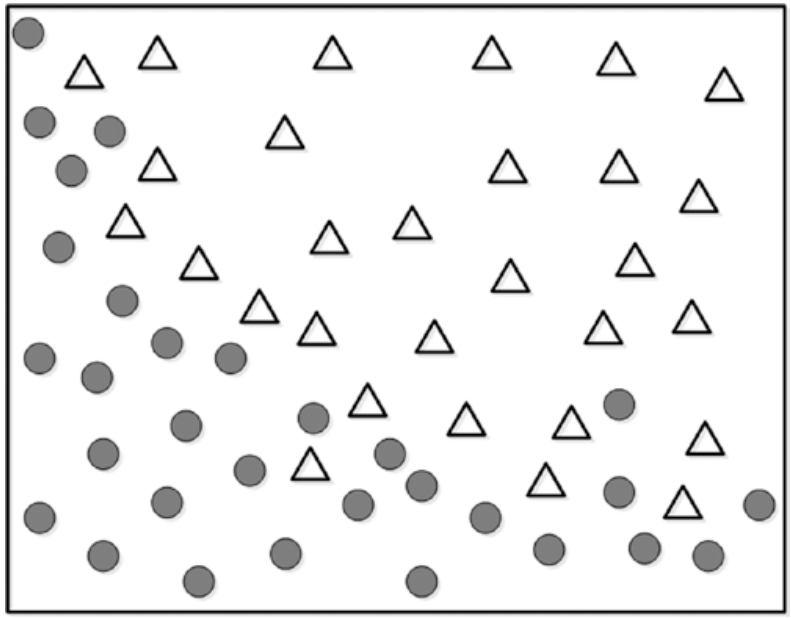
NN with Two Hidden Layers (HW)



Regularization (to avoid Overfitting)

One of the primary causes of corruption of the generalization process is overfitting.

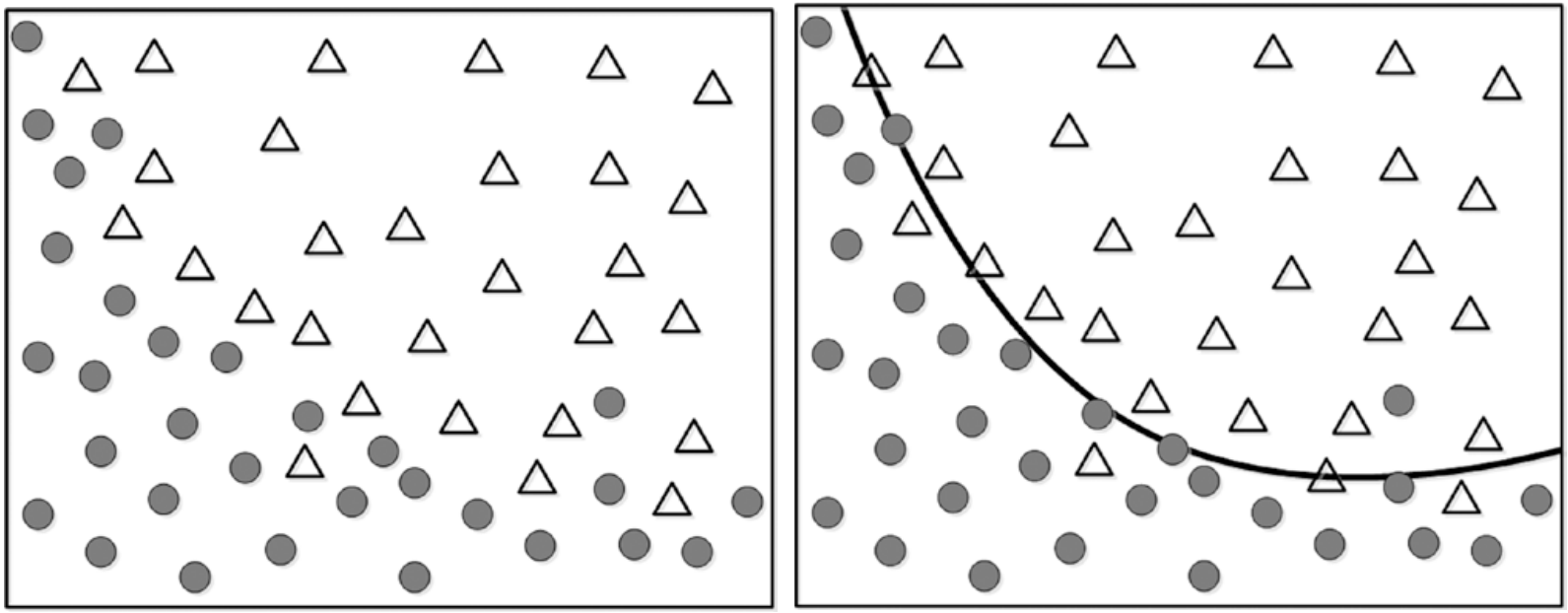
The objective is to determine a curve that defines the border of the two groups using the training data.



Overfitting

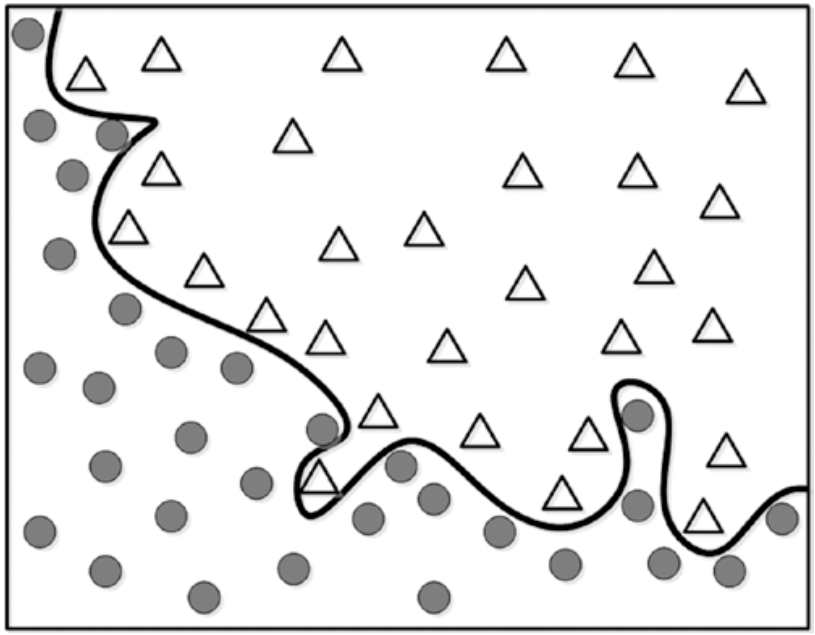
One of the primary causes of corruption of the generalization process is overfitting.

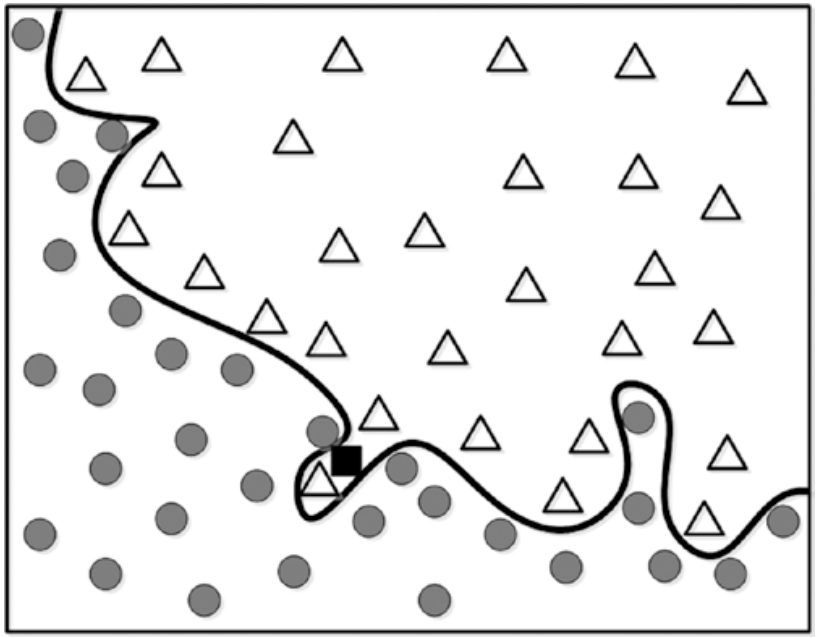
The objective is to determine a curve that defines the border of the two groups using the training data.



Overfitting

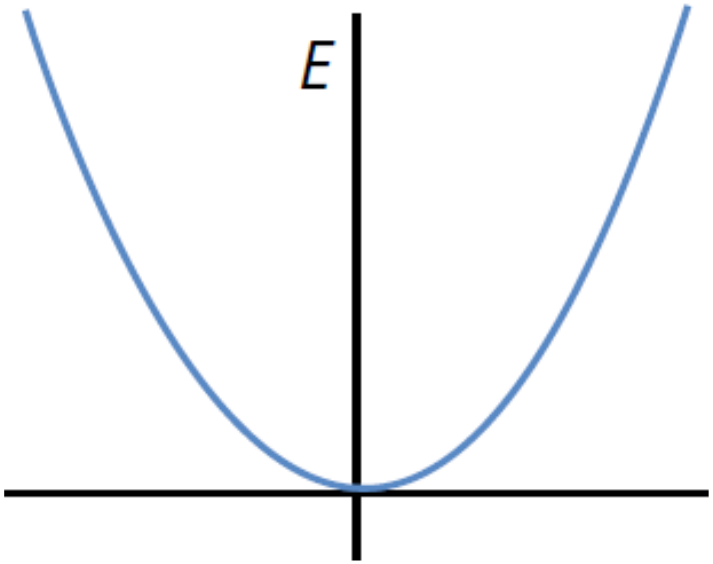
Some outliers penetrate the area of the other group and disturb the boundary. As Machine Learning considers all the data, even the noise, it ends up producing an improper model (a curve in this case). This would be penny-wise and pound-foolish.





Remedy : Regularization

- Regularization is a numerical method that attempts to construct a model structure as simple as possible. The simplified model can avoid the effects of overfitting at the small cost of performance.
- Cost function $E = \sum_{i=1}^{N}(t_i - y_i)^2$ Sum of squared errors



Remedy : Regularization

- For this reason, overfitting of the neural network can be improved by adding the sum of weights to the cost function, (new) Cost function $J = \frac{1}{2}\sum_{i=1}^{N}(t_i - y_i)^2 + \frac{1}{2}\left\|w_{ij}\right\|^2$
- In order to drop the value of the cost function, both the error and weight should be controlled to be as small as possible.
- However, if a weight becomes small enough, the associated nodes will be practically disconnected. As a result, unnecessary connections are eliminated, and the neural network becomes simpler.

Add L1 Regularization to XOR Network

- New Loss function $L_{new} = \frac{1}{2}\sum_{i=1}^{N}(t_i - y_i)^2 + \sum|w_i|$
- The gradient of the regularized loss w.r.t a weight w is:

$$\frac{\partial L_{new}}{\partial w} = \frac{\partial L}{\partial w} + sign(w_i)$$

Update rule for weights w is

$$w = w - \alpha\left(\frac{\partial L}{\partial w} + sign(w)\right)$$

Add L2 Regularization to XOR Network

- New Loss function $L_{new} = \frac{1}{2}\sum_{i=1}^{N}(t_i - y_i)^2 + \frac{1}{2}\|w\|^2$
- The gradient of the regularized loss w.r.t a weight w is:

$$\frac{\partial L_{new}}{\partial w} = \frac{\partial L}{\partial w} + w$$

Update rule for weights w is

$$w = w - \alpha\left(\frac{\partial L}{\partial w} + w\right)$$

XOR implementation with L1

```
 # Apply L2 regularization to weights
hidden_layer_weights += learning_rate *
(np.dot(hidden_layer_output.T,
output_layer_delta) - sign(hidden_layer_weights))

input_layer_weights += learning_rate *
(np.dot(inputs.T, hidden_layer_delta) -
            sign(input_layer_weights))

# Update biases (no regularization applied to
biases) hidden_layer_bias +=
np.sum(output_layer_delta, axis=0,
      keepdims=True) * learning_rate
input_layer_bias += np.sum(hidden_layer_delta,
axis=0,     keepdims=True) * learning_rate
```

XOR implementation with L2

```
 # Apply L2 regularization to weights
hidden_layer_weights += learning_rate *
(np.dot(hidden_layer_output.T,
output_layer_delta) - hidden_layer_weights)

input_layer_weights += learning_rate *
(np.dot(inputs.T, hidden_layer_delta) -
            input_layer_weights)

# Update biases (no regularization applied to
biases) hidden_layer_bias +=
np.sum(output_layer_delta, axis=0,
      keepdims=True) * learning_rate
input_layer_bias += np.sum(hidden_layer_delta,
axis=0,     keepdims=True) * learning_rate
```

Norm Penalties as Constrained Optimization

- Denote Regularized Objective function

$$J(\theta; X, y) = L(\theta; X, y) + \alpha\Omega(\theta)$$

$L(\theta; X, y)$ is a loss function e.g., MSE, Cross-entropy

$\Omega(\theta)$ is a penalty function.

$\alpha \epsilon [0, \infty)$ is hyerparameter. Setting α to 0 results in no regularization. Larger values of α correspond to more regularization.

- L^2 Norm Parameter Regularization (ridge regression or Tikhonov regularization)

If $\Omega(\theta) = \frac{1}{2}\|w\|_2^2$

- L^1 Norm Regularization

If $\Omega(\theta) = \|w\|_1 = \sum_i |w|$

Norm Penalties as Constrained Optimization

Consider the Objective function with L^2 Norm Regularization

$$J(\theta; X, y) = L(\theta; X, y) + \alpha \frac{1}{2} \|w\|_2^2$$

Transform the L2 regularization term into a constraint:

$$\min_{\theta} L(\theta; X, y)$$

Subject to $\sum W^2 \leq \tau$

Where τ>0: A constraint hyperparameter controlling the regularization strength.

- Larger τ: Weaker regularization (equivalent to smaller α).
- Smaller τ: Stronger regularization (equivalent to larger α).

Norm Penalties as Constrained Optimization

The constrained optimization can be reformulated using the Lagrangian multiplier λ

$$J(\theta; X, y) = L(\theta; X, y) + \lambda\left(\sum W^2 - \tau\right)$$

Here, λ acts as a penalty parameter for violating the constraint $\sum W^2 \leq \tau$

This formulation helps in using techniques like **dual optimization** or **projected gradient descent** to enforce constraints during optimization.

Norm Penalties as Constrained Optimization – Backpropagation

Consider $L = \frac{1}{N}\sum_{i=1}^{N}(y_i - t_i)^2 + \lambda(\sum W^2 - \tau)$

Containing the MSE and the constraint term

1. Compute Gradients for MSE Term

The gradients of the MSE term with respect to the weights and biases are:

$$\frac{\partial L_{MSE}}{\partial y_i} = \frac{2}{N}(y_i - t_i)$$

Propagate through the layers as in standard backpropagation.

2. Compute Gradients for L2 Constraint

The L2 regularization gradient is:

$$\frac{\partial\left(\lambda(\sum W^2 - \tau)\right)}{\partial w} = 2\lambda W$$

This term is added to the gradient from the MSE during weight updates.

Norm Penalties as Constrained Optimization – Backpropagation

Enforcing the Constraint ($\sum W^2 - \tau$)

To ensure the constraint is satisfied, after each weight update:

1. **Check the L^2 Norm**

$$\|w\|_2^2 = \sum W^2$$

2. Project Back if Necessary: If $\|w\|_2^2 > \tau$, rescale W as follows

$$W \leftarrow W . \sqrt{\frac{\tau}{\|w\|_2^2}}$$

Numerical Example

Consider two inputs, one hidden layer with two neurons and one output. NO bias for any neuron.

1. Let initial weights $W_h = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.1 \end{bmatrix}, W_o = \begin{bmatrix} 0.2 \\ -0.5 \end{bmatrix}$
2. Using back propagation, say we obtained $\Delta W_o = \begin{bmatrix} -0.014 \\ -0.020 \end{bmatrix}, \Delta W_h = \begin{bmatrix} 0.02 & -0.01 \\ -0.015 & 0.005 \end{bmatrix},$
3. For learning rate $\mu = 0.1$, update

$$W_o = W_o - \mu \Delta W_o$$
$$W_h = W_h - \mu \Delta W_h$$

Numerical Example

4. Updated, new weights $W_h = \begin{bmatrix} 0.498 & -0.299 \\ 0.8015 & 0.0995 \end{bmatrix}$, $W_o = \begin{bmatrix} 0.2014 \\ -0.498 \end{bmatrix}$
5. Compute Current L^2 Norm

$$\sum W^2 = \|W_h\|^2 + \|W_o\|^2$$

$$= 0.498^2 + (-0.299)^2 + 0.8015^2 + 0.0995^2 + 0.2014^2 + (-0.498)^2 = 1.251$$

6. If τ=1.5, the constraint $(\sum W^2 \leq \tau)$ is already satisfied, and no adjustment is required.

Numerical Example

7. If τ=1.5, the constraint $(\sum W^2 \leq \tau)$ is already satisfied, and no adjustment is required.
8. If τ=1.0, the constraint $(\sum W^2 \leq \tau)$ is violated. Then do the following adjustment

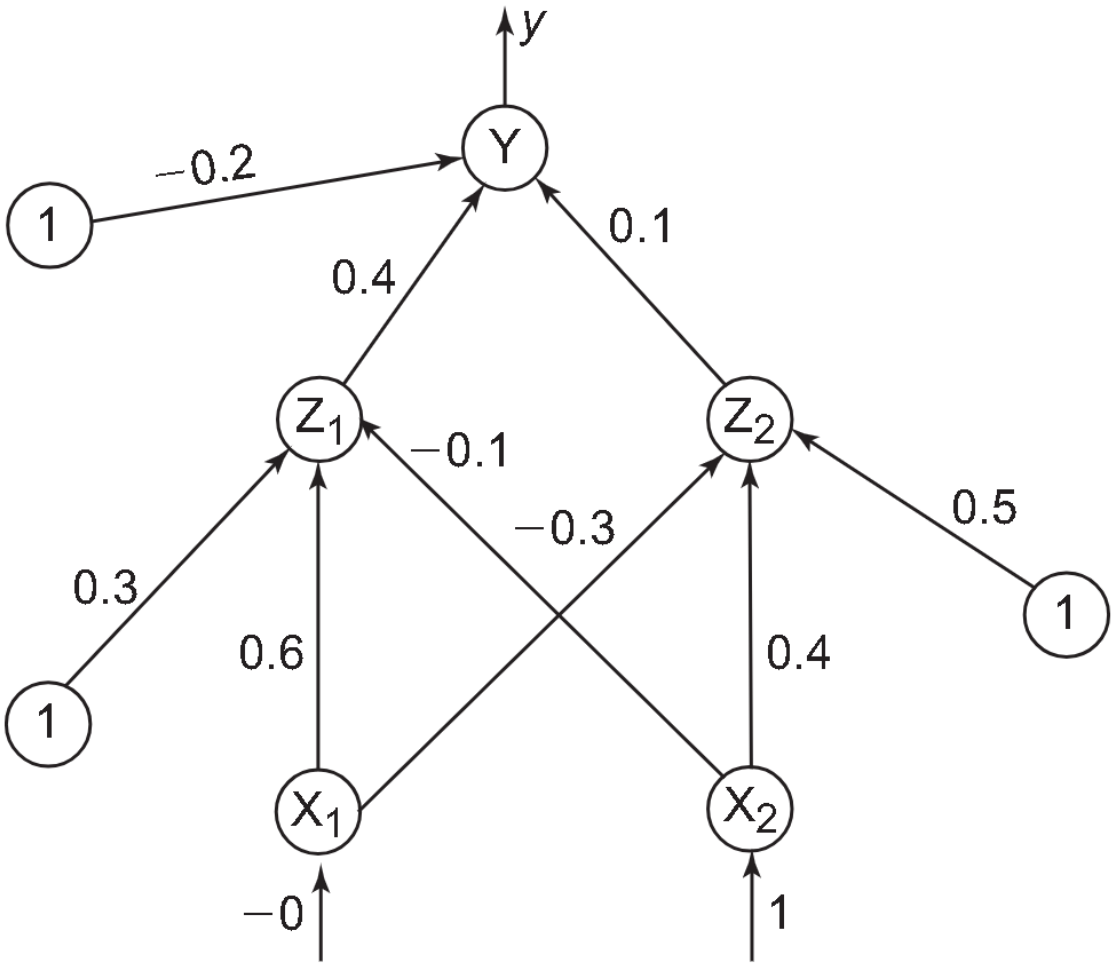
$$Scaling\ factor = \sqrt{\frac{\tau}{\|w\|_2^2}} = \sqrt{\frac{1}{1.251}} = 0.894$$

9. $W_h \leftarrow W_h.Scaling\ factor = \begin{bmatrix} 0.498 & -0.299 \\ 0.8015 & 0.0995 \end{bmatrix} 0.894$
10. $W_o \leftarrow W_o.Scaling\ factor = \begin{bmatrix} 0.2014 \\ -0.498 \end{bmatrix} 0.894$

Thank You All Very Much

Appendix: Example Implementation

Using Back-propagation network, find the new weights for the network shown aside. Input = [0 1] and target output is 1. use learning rate 0.25 and binary sigmoidal activation function



1. Consolidate the information

- Given: Inputs [0 1], target 1.
- $[v_{1,1} v_{2,1} v_{0,1}] = [0.6 \ -0.1 \ 0.3]$,
- $[v_{1,2} v_{2,2} v_{0,2}] = [-0.3 \ 0.4 \ 0.5]$
- $[w_1 w_2 w_0]$=[0.4 0.1 $-$ 0.2]
- Learning rate $\alpha = 0.25$
- Activation function is binary sigmoidal $f(x) = \frac{1}{1+e^{-x}}$
- Derivative $f'(x) = \frac{-1}{(1+e^{-x})^2}(-e^{-x})$

$$= \frac{1}{1+e^{-x}}\left[1 - \frac{1}{1+e^{-x}}\right] = f(x)(1 - f(x))$$

2. Feed-forward Phase

1. Hidden unit $z_j = \sum_{i=1}^{2} v_{i,j}\, x_i + v_{0,j}, j = 1,2$

   $z_1 = v_{1,1}\, x_1 + v_{2,1}\, x_2 + v_{0,1} =$

2. $z_2 = v_{1,2}\, x_1 + v_{2,2}\, x_2 + v_{0,2} =$

3. Output $Z_j = f\left(z_j\right)$, sigmoidal activation function

   $Z_1 = f(z_1) = \frac{1}{1+e^{-z_1}} =$

4. $Z_2 = f(z_2) = \frac{1}{1+e^{-z_2}} =$

5. Output unit $y = \sum_{j=1}^{2} w_j\, Z_j + w_0 =$

6. Output $Y = f(y_i)$ sigmoidal activation function

   $Y = f(y) = \frac{1}{1+e^{-y}} =$

2. Feed-forward Phase

1. Hidden unit $z_j = \sum_{i=1}^{2} v_{i,j}\, x_i + v_{0,j}, j = 1,2$

$$z_1 = v_{1,1}\, x_1 + v_{2,1}\, x_2 + v_{0,1} = 0.2$$

$$z_2 = v_{1,2}\, x_1 + v_{2,2}\, x_2 + v_{0,2} = 0.9$$

2. Output $Z_j = f(z_j)$, sigmoidal activation function

$$Z_1 = f(z_1) = \frac{1}{1+e^{-0.2}} = 0.5498, Z_2 = 0.7109$$

3. Output unit $y = \sum_{j=1}^{2} w_j\, Z_j + w_0 = 0.091$

4. Output $Y = f(y_i)$ sigmoidal activation function

$$Y = f(y) = \frac{1}{1+e^{-0.091}} = 0.5227$$

3. Back-propagation of error Phase

5. Compute error correction term

$$\delta_k = (t_k - Y)f'(y_i) = (t - Y)f(y)[1 - f(y)] =$$

6. Compute change in weights and bias

$$\Delta w_{j,k} = \delta \quad Z_j \,, \qquad \Delta w_{0,k} = \delta$$

$$\Delta w_1 = \delta \quad Z_1 =,$$

$$\Delta w_2 = \delta \quad Z_2 =,$$

$$\Delta w_0 = \delta \quad =,$$

3. Back-propagation of error Phase

5. Compute error correction term

$$\delta_k = (t_k - Y)f'(y_i) = (t - Y)f(y)[1 - f(y)]$$

$$\delta = (1 - 0.5227)0.5227(1 - 0.5227) = 0.1191$$

6. Compute change in weights and bias

$$\Delta w_{j,k} = \delta \quad Z_j \,, \qquad \Delta w_{0,k} = \delta$$

$$\alpha\Delta w_1 = 0.25 \times 0.1191 \times 0.5498 = 0.0164,$$

$$\alpha\Delta w_2 = 0.25 \times 0.1191 \times 0.7109 = 0.02117,$$

$$\alpha\Delta w_0 = 0.25 \times 0.1191 = 0.02978,$$

7. Hidden unit $\delta_{i,j} = \sum_{k=1}^{m} \delta_k w_{j,k}$

$$\delta_{1,1} = \delta w_1 =$$

$$\delta_{1,2} = \delta w_2 =$$

3. Back-propagation of error Phase

5. Compute error correction term

$$\delta_k = (t_k - Y)f'(y_i) = (t - Y)f(y)[1 - f(y)]$$

$$\delta = (1 - 0.5227)0.5227(1 - 0.5227) = 0.1191$$

6. Compute change in weights and bias

$$\Delta w_{j,k} = \delta \quad Z_j \,, \qquad \Delta w_{0,k} = \delta$$

$$\alpha\Delta w_1 = 0.25 \times 0.1191 \times 0.5498 = 0.0164,$$

$$\alpha\Delta w_2 = 0.25 \times 0.1191 \times 0.7109 = 0.02117,$$

$$\alpha\Delta w_0 = 0.25 \times 0.1191 = 0.02978,$$

7. Hidden unit $\delta_{i,j} = \sum_{k=1}^{m} \delta_k w_{j,k}$

$$\delta_{1,1} = \delta w_1 = 0.1191 \times 0.4 = 0.04764$$

$$\delta_{1,2} = \delta w_2 = 0.1191 \times 0.1 = 0.01191$$

3. Back-propagation of error Phase

8. Calculate error term $\delta_j = \delta_{i,j} f'(z_j)$

$$\delta_1 = \delta_{1,1} f'(z_1) = \delta_{1,1} f(z_1)[1 - f(z_1)] =$$

$$\delta_2 = \delta_{1,2} f(z_2)[1 - f(z_2)] =$$

3. Back-propagation of error Phase

8. Calculate error term $\delta_j = \delta_{i,j} f'(z_j)$

$$\delta_1 = \delta_{1,1} f'(z_1) = 0.04764 \times 0.5498[1 - 0.5498] = 0.0118$$

$$\delta_2 = \delta_{1,2} f'(z_2) = 0.01191 \times 0.7109[1 - 0.7109] = 0.00245$$

3. Back-propagation of error Phase

8. Calculate error term $\delta_j = \delta_{i,j} f'(z_j)$

   $\delta_1 = 0.0118$ $\delta_2 = 0.00245$

9. Compute change in weights and bias

$\Delta v_{i,j} = \delta_j x_i,$ $\Delta v_{0,j} = \delta_j$

$$\Delta v_{1,1} = \delta_1 x_1 =$$
$$\Delta v_{2,1} = \delta_1 x_2 =$$
$$\Delta v_{0,1} = \delta_1 =$$

$$\Delta v_{1,2} = \delta_2 x_1 =$$
$$\Delta v_{2,2} = \delta_2 x_2 =$$
$$\Delta v_{0,2} = \delta_2 =$$

3. Back-propagation of error Phase

8. Calculate error term $\delta_j = \delta_{i,j} f'(z_j)$

   $\delta_1 = 0.0118$ $\delta_2 = 0.00245$

9. Compute change in weights and bias

   $\Delta v_{i,j} = \delta_j x_i,$ $\Delta v_{0,j} = \delta_j$

$\alpha \Delta v_{1,1} = \alpha \delta_1 x_1 = 0.25 \times 0.0118 \times 0 = 0$

$\alpha \Delta v_{2,1} = \alpha \delta_1 x_2 = 0.25 \times 0.0118 \times 1 = 0.00295$

$\alpha \Delta v_{0,1} = \alpha \delta_1 = 0.25 \times 0.0118 = 0.00295$

$\alpha \Delta v_{1,2} = \alpha \delta_2 x_1 =$

$\alpha \Delta v_{2,2} = \delta_2 x_2 =$

$\alpha \Delta v_{0,2} = \delta_2 =$

3. Back-propagation of error Phase

8. Calculate error term $\delta_j = \delta_{i,j} f'(z_j)$

   $\delta_1 = 0.0118$ $\delta_2 = 0.00245$

9. Compute change in weights and bias

   $\Delta v_{i,j} = \alpha \delta_j x_i,$ $\Delta v_{0,j} = \alpha \delta_j$

$\alpha \Delta v_{1,1} = \alpha \delta_1 x_1 = 0.25 \times 0.0118 \times 0 = 0$

$\alpha \Delta v_{2,1} = \alpha \delta_1 x_2 = 0.25 \times 0.0118 \times 1 = 0.00295$

$\alpha \Delta v_{0,1} = \alpha \delta_1 = 0.25 \times 0.0118 = 0.00295$

$\alpha \Delta v_{1,2} = \alpha \delta_2 x_1 = 0.25 \times 0.00245 \times 0 = 0$

$\alpha \Delta v_{2,2} = \alpha \delta_2 x_2 = 0.25 \times 0.00245 \times 1 = 0.00061255$

$\alpha \Delta v_{0,2} = \alpha \delta_2 = 0.25 \times 0.00245 = 0.00061255$

3. Back-propagation of error Phase

8. Calculate error term $\delta_j = \delta_{i,j} f'(z_j)$

$$\delta_1 = \delta_{1,1} f'(z_1) = 0.04764 \times 0.5498[1 - 0.5498] = 0.0118$$

$$\delta_2 = \delta_{1,2} f'(z_2) = 0.01191 \times 0.7109[1 - 0.7109] = 0.00245$$

9. Compute change in weights and bias

$$\Delta v_{i,j} = \delta_j x_i, \qquad \Delta v_{0,j} = \delta_j$$

$$\alpha \Delta v_{1,1} = \alpha \delta_1 x_1 = 0.25 \times 0.0118 \times 0 = 0$$

$$\alpha \Delta v_{2,1} = \alpha \delta_1 x_2 = 0.25 \times 0.0118 \times 1 = 0.00295$$

$$\alpha \Delta v_{0,1} = \alpha \delta_1 = 0.25 \times 0.0118 = 0.00295$$

$$\alpha \Delta v_{1,2} = \alpha \delta_2 x_1 = 0.25 \times 0.00245 \times 0 = 0$$

$$\alpha \Delta v_{2,2} = \alpha \delta_2 x_2 = 0.25 \times 0.00245 \times 1 = 0.00061255$$

$$\alpha \Delta v_{0,2} = \alpha \delta_2 = 0.25 \times 0.00245 = 0.00061255$$

4. Weights and Bias update phase

10. Each output unit, $k = 1$ to m update weights and bias

$$w_{j,k}^{new} = w_{j,k}^{old} + \alpha \Delta w_{j,k} \ ,$$

$$w_{0,k}^{new} = w_{0,k}^{old} + \alpha \Delta w_{0,k}$$

$$w_1^{new} = w_1^{old} + \alpha \Delta w_1 =$$

$$w_2^{new} = w_2^{old} + \alpha \Delta w_2 =$$

$$w_0^{new} = w_0^{old} + \alpha \Delta w_0 =$$

4. Weights and Bias update phase

10. Each output unit, $k = 1$ to m update weights and bias

$$w_{j,k}^{new} = w_{j,k}^{old} + \alpha \Delta w_{j,k} \ ,$$

$$w_{0,k}^{new} = w_{0,k}^{old} + \alpha \Delta w_{0,k}$$

$$w_1^{new} = w_1^{old} + \alpha \Delta w_1 = 0.4 + 0.0164 = 0.4164$$

$$w_2^{new} = w_2^{old} + \alpha \Delta w_2 = 0.1 + 0.02117 = 0.12117$$

$$w_0^{new} = w_0^{old} + \alpha \Delta w_0 = -0.2 + 0.02978 = -0.17022$$

4. Weights and Bias update phase

11. Each hidden unit, j = 1 to p update weights and bias

$$v_{i,j}^{new} = v_{i,j}^{old} + \alpha \Delta v_{i,j} \qquad v_{0,j}^{new} = v_{0,j}^{old} + \alpha \Delta v_{0,j}$$

$$v_{0,1}^{new} = v_{0,1}^{old} + \alpha \Delta v_{0,1} =$$

$$v_{1,1}^{new} = v_{1,1}^{old} + \alpha \Delta v_{1,1} =$$

$$v_{2,1}^{new} = v_{2,1}^{old} + \alpha \Delta v_{2,1} =$$

$$v_{0,2}^{new} = v_{0,2}^{old} + \alpha \Delta v_{0,2} =$$

$$v_{1,2}^{new} = v_{1,2}^{old} + \alpha \Delta v_{1,2} =$$

$$v_{2,2}^{new} = v_{2,2}^{old} + \alpha \Delta v_{2,2} =$$

4. Weights and Bias update phase

11. Each hidden unit, j = 1 to p update weights and bias

$$v_{i,j}^{new} = v_{i,j}^{old} + \alpha \Delta v_{i,j} \qquad v_{0,j}^{new} = v_{0,j}^{old} + \alpha \Delta v_{0,j}$$

$$v_{0,1}^{new} = v_{0,1}^{old} + \alpha \Delta v_{0,1} =$$

$$v_{1,1}^{new} = v_{1,1}^{old} + \alpha \Delta v_{1,1} =$$

$$v_{2,1}^{new} = v_{2,1}^{old} + \alpha \Delta v_{2,1} =$$

$$v_{0,2}^{new} = v_{0,2}^{old} + \alpha \Delta v_{0,2} = 0.5 + 0.0006125 = 0.5006125$$

$$v_{1,2}^{new} = v_{1,2}^{old} + \alpha \Delta v_{1,2} = -0.3 + 0 = -0.3$$

$$v_{2,2}^{new} = v_{2,2}^{old} + \alpha \Delta v_{2,2} = 0.4 + 0.0006125 = 0.4006125$$

4. Weights and Bias update phase

11. Each hidden unit, $j = 1$ to p update weights and bias

$$v_{i,j}^{new} = v_{i,j}^{old} + \alpha \Delta v_{i,j} \qquad v_{0,j}^{new} = v_{0,j}^{old} + \alpha \Delta v_{0,j}$$

$$v_{0,1}^{new} = v_{0,1}^{old} + \alpha \Delta v_{0,1} = 0.3 + 0.00295 = 0.30295$$

$$v_{1,1}^{new} = v_{1,1}^{old} + \alpha \Delta v_{1,1} = 0.6 + 0 = 0.6$$

$$v_{2,1}^{new} = v_{2,1}^{old} + \alpha \Delta v_{2,1} = -0.1 + 0.00295 = -0.09705$$

$$v_{0,2}^{new} = v_{0,2}^{old} + \alpha \Delta v_{0,2} = 0.5 + 0.0006125 = 0.5006125$$

$$v_{1,2}^{new} = v_{1,2}^{old} + \alpha \Delta v_{1,2} = -0.3 + 0 = -0.3$$

$$v_{2,2}^{new} = v_{2,2}^{old} + \alpha \Delta v_{2,2} = 0.4 + 0.0006125 = 0.4006125$$

Epoch	v_{11}	v_{21}	v_{01}	v_{12}	v_{22}	v_{02}
0	0.6	-0.1	0.3	-0.3	0.4	0.5
1	0.6	-0.097	0.303	-0.3	0.401	0.501

Epoch	z_1	z_2	w_1	w_2	w_0	y
0	0.549	0.711	0.4	0.1	-0.2	0.523
1	0.5513	0.7113	0.416	0.121	-0.17	0.5363

Write a program for this case and cross-verify your answers. After how many epochs will the output converge?